
Chapter 13 GUI Programming

Topics

13.1 Graphical User Interfaces 

13.2 Using the `tkinter` Module 

13.3 Display Text with `Label` Widgets 

13.4 Organizing Widgets with `Frames` 

13.5 `Button` Widgets and Info Dialog Boxes 

13.6 Getting Input with the `Entry` Widget 

13.7 Using Labels as Output Fields 

13.8 Radio Buttons and Check Buttons 

13.9 Drawing Shapes with the `Canvas` Widget 

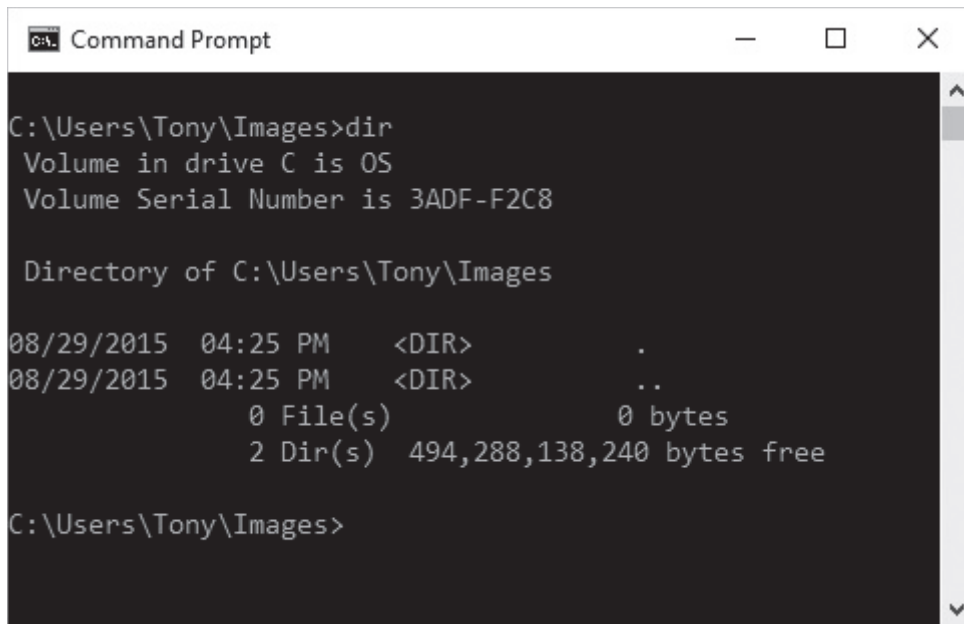
13.1 Graphical User Interfaces

Concept:

A graphical user interface allows the user to interact with the operating system and other programs using graphical elements such as icons, buttons, and dialog boxes.

A computer's *user interface* is the part of the computer with which the user interacts. One part of the user interface consists of hardware devices, such as the keyboard and the video display. Another part of the user interface lies in the way that the computer's operating system accepts commands from the user. For many years, the only way that the user could interact with an operating system was through a *command line interface*, such as the one shown in [Figure 13-1](#) . A command line interface typically displays a prompt, and the user types a command, which is then executed.

Figure 13-1 A command line interface



```
Command Prompt

C:\Users\Tony\Images>dir
Volume in drive C is OS
Volume Serial Number is 3ADF-F2C8

Directory of C:\Users\Tony\Images

08/29/2015  04:25 PM    <DIR>          .
08/29/2015  04:25 PM    <DIR>          ..
               0 File(s)                0 bytes
               2 Dir(s)  494,288,138,240 bytes free

C:\Users\Tony\Images>
```

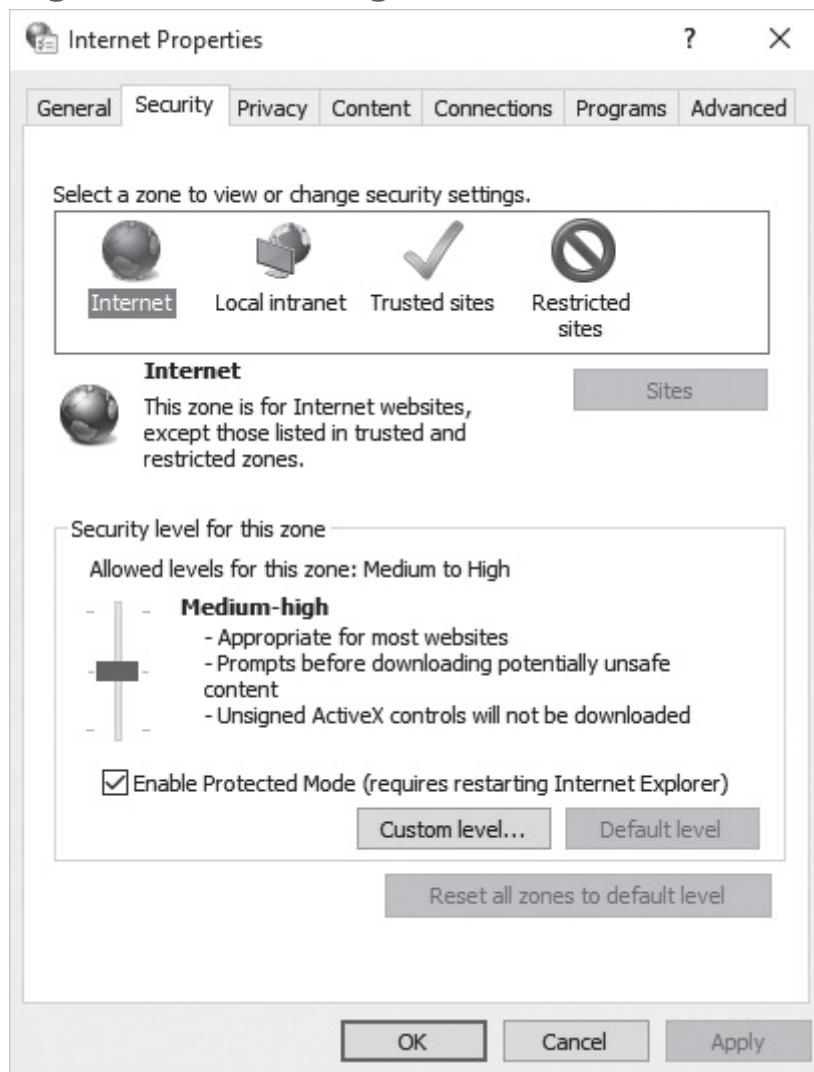
Many computer users, especially beginners, find command line interfaces difficult to use. This is because there are many commands to be learned, and each command has its own syntax, much like a programming statement. If a command isn't entered correctly, it will not work.

In the 1980s, a new type of interface known as a graphical user interface came into use in commercial operating systems. A *graphical user interface (GUI*; pronounced "gooey"), allows the user to interact with the operating system and other programs through graphical elements on the screen. GUIs also popularized the use of the mouse as an input device. Instead of requiring the user to type commands on the keyboard, GUIs allow the user to point at graphical elements and click the mouse button to activate them.

Much of the interaction with a GUI is done through *dialog boxes*, which are small windows that display information and allow the user to

perform actions. **Figure 13-2**  shows an example of a dialog box from the Windows operating system that allows the user to change the system's Internet settings. Instead of typing commands according to a specified syntax, the user interacts with graphical elements such as icons, buttons, and slider bars.

Figure 13-2 A dialog box

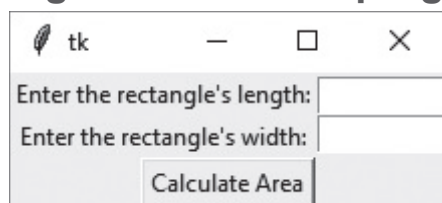


GUI Programs Are Event-Driven

In a text-based environment such as a command line interface, programs determine the order in which things happen. For example, consider a program that calculates the area of a rectangle. First, the program prompts the user to enter the rectangle's width. The user enters the width, then the program prompts the user to enter the rectangle's length. The user enters the length, then the program calculates the area. The user has no choice but to enter the data in the order that it is requested.

In a GUI environment, however, the user determines the order in which things happen. For example, **Figure 13-3**  shows a GUI program (written in Python) that calculates the area of a rectangle. The user can enter the length and the width in any order he or she wishes. If a mistake is made, the user can delete the data that was entered and retype it. When the user is ready to calculate the area, he or she clicks the *Calculate Area* button, and the program performs the calculation. Because GUI programs must respond to the actions of the user, it is said that they are *event-driven*. The user causes events to take place, such as the clicking of a button, and the program must respond to the events.

Figure 13-3 A GUI program



Checkpoint

13.1 What is a user interface?

13.2 How does a command line interface work?

13.3 When the user runs a program in a text-based environment, such as the command line, what determines the order in which things happen?

13.4 What is an event-driven program?

13.2 Using the `tkinter` Module

Concept:

In Python, you can use the `tkinter` module to create simple GUI programs.

Python does not have GUI programming features built into the language itself. However, it comes with a module named `tkinter` that allows you to create simple GUI programs. The name “tkinter” is short for “Tk interface.” It is named this because it provides a way for Python programmers to use a GUI library named Tk. Many other programming languages use the Tk library as well.



Note:

There are numerous GUI libraries available for Python. Because the `tkinter` module comes with Python, we will use it only in this chapter.

A GUI program presents a window with various graphical *widgets* with which the user can interact or view. The `tkinter` module provides 15 widgets, which are described in [Table 13-1](#) . We won't cover all of the `tkinter` widgets in this chapter, but we will demonstrate how to create simple GUI programs that gather input and display data.

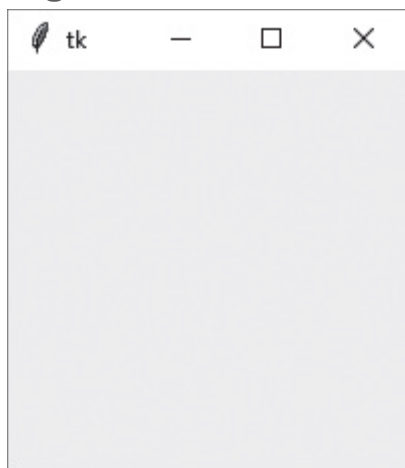
Table 13-1 `tkinter` widgets

Widget	Description
<code>Button</code>	A button that can cause an action to occur when it is clicked.
<code>Canvas</code>	A rectangular area that can be used to display graphics.
<code>Checkbutton</code>	A button that may be in either the “on” or “off” position.
<code>Entry</code>	An area in which the user may type a single line of input from the keyboard.
<code>Frame</code>	A container that can hold other widgets.
<code>Label</code>	An area that displays one line of text or an image.
<code>Listbox</code>	A list from which the user may select an item
<code>Menu</code>	A list of menu choices that are displayed when the user clicks a <code>Menubutton</code> widget.
<code>Menubutton</code>	A menu that is displayed on the screen and may be clicked by the user
<code>Message</code>	Displays multiple lines of text.

<code>Radiobutton</code>	A widget that can be either selected or deselected. <code>Radiobutton</code> widgets usually appear in groups and allow the user to select one of several options.
<code>Scale</code>	A widget that allows the user to select a value by moving a slider along a track.
<code>Scrollbar</code>	Can be used with some other types of widgets to provide scrolling ability.
<code>Text</code>	A widget that allows the user to enter multiple lines of text input.
<code>Toplevel</code>	A container, like a <code>Frame</code> , but displayed in its own window.

The simplest GUI program that we can demonstrate is one that displays an empty window. **Program 13-1**  shows how we can do this using the `tkinter` module. When the program runs, the window shown in **Figure 13-4**  is displayed. To exit the program, simply click the standard Windows close button (×) in the upper right corner of the window.

Figure 13-4 Window displayed by **Program 13-1** 





Note:

Programs that use `tkinter` do not always run reliably under IDLE. This is because IDLE itself uses `tkinter`. You can always use IDLE's editor to write GUI programs, but for the best results, run them from your operating system's command prompt.

Program 13-1 (empty_window1.py)

```
1  # This program displays an empty window.
2
3  import tkinter
4
5  def main():
6      # Create the main window widget.
7      main_window = tkinter.Tk()
8
9      # Enter the tkinter main loop.
10     tkinter.mainloop()
11
12 # Call the main function.
13 main()
```

Line 3 imports the `tkinter` module. Inside the `main` function, line 7 creates an instance of the `tkinter` module's `Tk` class and assigns it to the `main_window` variable. This object is the root widget, which is the main window in the program. Line 10 calls the `tkinter` module's `mainloop` function. This function runs like an infinite loop until you close the main window.

Most programmers prefer to take an object-oriented approach when writing a GUI program. Rather than writing a function to create the on-screen elements of a program, it is a common practice to write a class with an `__init__` method that builds the GUI. When an instance of the class is created, the GUI appears on the screen. To demonstrate, **Program 13-2**  shows an object-oriented version of our program that displays an empty window. When this program runs it displays the window shown in **Figure 13-4** .

Program 13-2 `(empty_window2.py)`

```
1  # This program displays an empty window.
2
3  import tkinter
4
5  class MyGUI:
6      def __init__(self):
7          # Create the main window widget.
8          self.main_window = tkinter.Tk()
9
```

```
10     # Enter the tkinter main loop.
11     tkinter.mainloop()
12
13     # Create an instance of the MyGUI class.
14     my_gui = MyGUI()
```

Lines 5 through 11 are the class definition for the `MyGUI` class. The class's `__init__` method begins in line 6. Line 8 creates the root widget and assigns it to the class attribute `main_window`. Line 11 executes the `tkinter` module's `mainloop` function. The statement in line 14 creates an instance of the `MyGUI` class. This causes the class's `__init__` method to execute, displaying the empty window on the screen.



Checkpoint

13.5 Briefly describe each of the following `tkinter` widgets:

- a. `Label`
- b. `Entry`
- c. `Button`
- d. `Frame`

13.6 How do you create a root widget?

13.7 What does the `tkinter` module's `mainloop` function do?

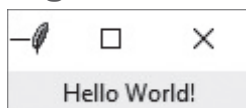
13.3 Display Text with `Label` Widgets

Concept:

You use the `Label` widget to display text in a window.

You can use a `Label` widget to display a single line of text in a window. To make a `Label` widget you create an instance of the `tkinter` module's `Label` class. **Program 13-3**  creates a window containing a `Label` widget that displays the text “Hello World!” The window is shown in **Figure 13-5** .

Figure 13-5 Window displayed by **Program 13-3** 



Creating a Simple GUI application

Program 13-3 (hello_world.py)

```
1  # This program displays a label with text.
2
3  import tkinter
4
5  class MyGUI:
6      def __init__(self):
7          # Create the main window widget.
8          self.main_window = tkinter.Tk()
9
10         # Create a Label widget containing the
11         # text 'Hello World!'
12         self.label = tkinter.Label(self.main_window,
13                                     text='Hello World!')
14
15         # Call the Label widget's pack method.
16         self.label.pack()
17
18         # Enter the tkinter main loop.
19         tkinter.mainloop()
20
21 # Create an instance of the MyGUI class.
22 my_gui = MyGUI()
```

The `MyGUI` class in this program is very similar to the one you saw previously in [Program 13-2](#). Its `__init__` method builds the GUI when an instance of the class is created. Line 8 creates a root widget and assigns it to `self.main_window`. The following statement appears in lines 12 and 13:

```
self.label = tkinter.Label(self.main_window,  
                           text='Hello World!')
```

This statement creates a `Label` widget and assigns it to `self.label`. The first argument inside the parentheses is `self.main_window`, which is a reference to the root widget. This simply specifies that we want the `Label` widget to belong to the root widget. The second argument is `text='Hello World!'`. This specifies the text that we want to be displayed in the label.

The statement in line 16 calls the `Label` widget's `pack` method. The `pack` method determines where a widget should be positioned and makes the widget visible when the main window is displayed. (You call the `pack` method for each widget in a window.) Line 19 calls the `tkinter` module's `mainloop` method, which displays the program's main window, shown in [Figure 13-5](#).

Let's look at another example. **Program 13-4**  displays a window with two `Label` widgets, shown in **Figure 13-6** .

Figure 13-6 Window displayed by **Program 13-4** 



Program 13-4 (`hello_world2.py`)

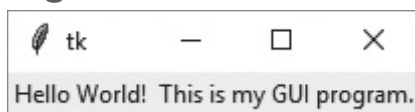
```
1  # This program displays two labels with text.
2
3  import tkinter
4
5  class MyGUI:
6      def __init__(self):
7          # Create the main window widget.
8          self.main_window = tkinter.Tk()
9
10         # Create two Label widgets.
11         self.label1 = tkinter.Label(self.main_window,
12                                     text='Hello World!')
13         self.label2 = tkinter.Label(self.main_window,
14                                     text='This is my GUI program.')
15
16         # Call both Label widgets' pack method.
17         self.label1.pack()
18         self.label2.pack()
```



```
19
20     # Enter the tkinter main loop.
21     tkinter.mainloop()
22
23     # Create an instance of the MyGUI class.
24     my_gui = MyGUI()
```

Notice the two `Label` widgets are displayed with one stacked on top of the other. We can change this layout by specifying an argument to `pack` method, as shown in [Program 13-5](#). When the program runs, it displays the window shown in [Figure 13-7](#).

Figure 13-7 Window displayed by [Program 13-5](#)



Program 13-5 (`hello_world3.py`)

```
1  # This program uses the side='left' argument with
2  # the pack method to change the layout of the widgets.
3
4  import tkinter
5
6  class MyGUI:
7      def __init__(self):
8          # Create the main window widget.
9          self.main_window = tkinter.Tk()
```

```

10
11         # Create two Label widgets.
12         self.label1 = tkinter.Label(self.main_window,
13                                     text='Hello World!')
14         self.label2 = tkinter.Label(self.main_window,
15                                     text='This is my GUI program.')
16
17         # Call both Label widgets' pack method.
18         self.label1.pack(side='left')
19         self.label2.pack(side='left')
20
21         # Enter the tkinter main loop.
22         tkinter.mainloop()
23
24     # Create an instance of the MyGUI class.
25     my_gui = MyGUI()

```

In lines 18 and 19, we call each `Label` widget's `pack` method passing the argument `side='left'`. This specifies that the widget should be positioned as far left as possible inside the parent widget. Because the `label1` widget was added to the `main_window` first, it will appear at the leftmost edge. The `label2` widget was added next, so it appears next to the `label1` widget. As a result, the labels appear side by side. The valid `side` arguments that you can pass to the `pack` method are `side='top'`, `side='bottom'`, `side='left'`, and `side='right'`.



Checkpoint

13.8 What does a widget's `pack` method do?

13.9 If you create two `Label` widgets and call their `pack` methods with no arguments, how will the `Label` widgets be arranged inside their parent widget?

13.10 What argument would you pass to a widget's `pack` method to specify that it should be positioned as far left as possible inside the parent widget?

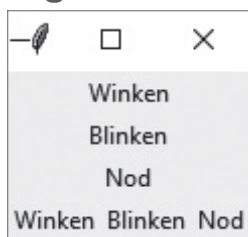
13.4 Organizing Widgets with Frames

Concept:

A Frame is a container that can hold other widgets. You can use Frames to organize the widgets in a window.

A `Frame` is a container. It is a widget that can hold other widgets. `Frame`s are useful for organizing and arranging groups of widgets in a window. For example, you can place a set of widgets in one `Frame` and arrange them in a particular way, then place a set of widgets in another `Frame` and arrange them in a different way. [Program 13-6](#)  demonstrates this. When the program runs it, displays the window shown in [Figure 13-8](#) .

Figure 13-8 Window displayed by [Program 13-6](#) 



Program 13-6 (frame_demo.py)

```
1  # This program creates labels in two different frames.
2
3  import tkinter
4
5  class MyGUI:
6      def __init__(self):
7          # Create the main window widget.
8          self.main_window = tkinter.Tk()
9
10         # Create two frames, one for the top of the
11         # window, and one for the bottom.
12         self.top_frame = tkinter.Frame(self.main_window)
13         self.bottom_frame =
tkinter.Frame(self.main_window)
14
15         # Create three Label widgets for the
16         # top frame.
17         self.label1 = tkinter.Label(self.top_frame,
18                                     text='Winken')
19         self.label2 = tkinter.Label(self.top_frame,
20                                     text='Blinken')
21         self.label3 = tkinter.Label(self.top_frame,
22                                     text='Nod')
23
24         # Pack the labels that are in the top frame.
25         # Use the side='top' argument to stack them
```

```
26         # one on top of the other.
27         self.label1.pack(side='top')
28         self.label2.pack(side='top')
29         self.label3.pack(side='top')
30
31         # Create three Label widgets for the
32         # bottom frame.
33         self.label4 = tkinter.Label(self.bottom_frame,
34                                     text='Winken')
35         self.label5 = tkinter.Label(self.bottom_frame,
36                                     text='Blinken')
37         self.label6 = tkinter.Label(self.bottom_frame,
38                                     text='Nod')
39
40         # Pack the labels that are in the bottom frame.
41         # Use the side='left' argument to arrange them
42         # horizontally from the left of the frame.
43         self.label4.pack(side='left')
44         self.label5.pack(side='left')
45         self.label6.pack(side='left')
46
47         # Yes, we have to pack the frames too!
48         self.top_frame.pack()
49         self.bottom_frame.pack()
50
51         # Enter the tkinter main loop.
52         tkinter.mainloop()
53
```

```
54 # Create an instance of the MyGUI class.  
55 my_gui = MyGUI()
```

Take a closer look at lines 12 and 13:

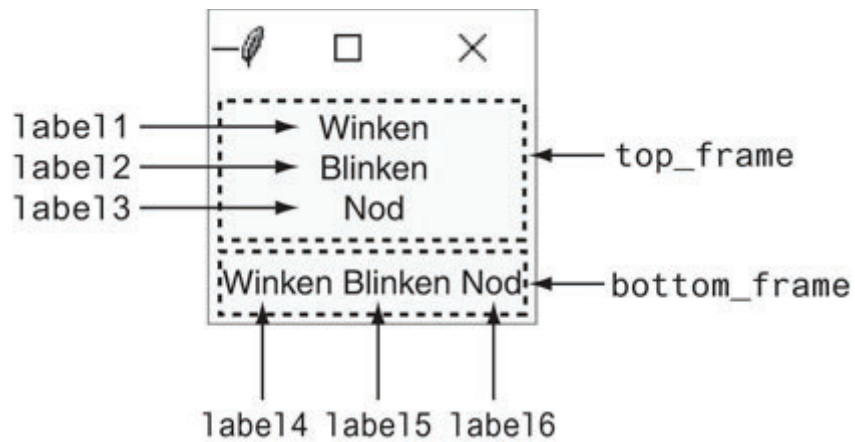
```
self.top_frame = tkinter.Frame(self.main_window)  
self.bottom_frame = tkinter.Frame(self.main_window)
```

These lines create two `Frame` objects. The `self.main_window` argument that appears inside the parentheses cause the `Frames` to be added to the `main_window` widget.

Lines 17 through 22 create three `Label` widgets. Notice these widgets are added to the `self.top_frame` widget. Then, lines 27 through 29 call each of the `Label` widgets' `pack` method, passing `side='top'` as an argument. As shown in [Figure 13-6](#) , this causes the three widgets to be stacked one on top of the other inside the `Frame`.

Lines 33 through 38 create three more `Label` widgets. These `Label` widgets are added to the `self.bottom_frame` widget. Then, lines 43 through 45 call each of the `Label` widgets' `pack` method, passing `side='left'` as an argument. As shown in [Figure 13-9](#) , this causes the three widgets to appear horizontally inside the `Frame`.

Figure 13-9 Arrangement of widgets



Lines 48 and 49 call the `Frame` widgets' `pack` method, which makes the `Frame` widgets visible. Line 52 executes the `tkinter` module's `mainloop` function.

13.5 Button Widgets and Info Dialog Boxes

Concept:

You use the Button widget to create a standard button in a window. When the user clicks a button, a specified function or method is called.

An info dialog box is a simple window that displays a message to the user, and has an OK button that dismisses the dialog box. You can use the `tkinter.messagebox` module's `showinfo` function to display an info dialog box.



Responding to Button Clicks

A `Button` is a widget that the user can click to cause an action to take place. When you create a `Button` widget you can specify the text that is to appear on the face of the button and the name of a callback function. A *callback function* is a function or method that executes when the user clicks the button.



Note:

A callback function is also known as an *event handler* because it handles the event that occurs when the user clicks the button.

To demonstrate, we will look at **Program 13-7** . This program displays the window shown in **Figure 13-10** . When the user clicks the button, the program displays a separate *info dialog box*, shown in **Figure 13-11** . We use a function named `showinfo`, which is in the `tkinter.messagebox` module, to display the info dialog box. (To use the `showinfo` function, you will need to import the `tkinter.messagebox` module.) This is the general format of the `showinfo` function call:

Figure 13-10 The main window displayed by **Program 13-7** 

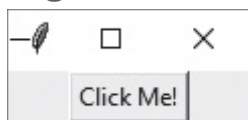
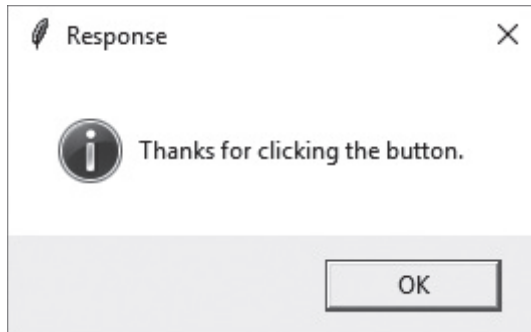


Figure 13-11 The info dialog box displayed by **Program 13-7** 



```
tkinter.messagebox.showinfo(title, message)
```

In the general format, `title` is a string that is displayed in the dialog box's title bar, and `message` is an informational string that is displayed in the main part of the dialog box.

Program 13-7 (`button_demo.py`)

```
1  # This program demonstrates a Button widget.
2  # When the user clicks the Button, an
3  # info dialog box is displayed.
4
5  import tkinter
6  import tkinter.messagebox
7
8  class MyGUI:
9      def __init__(self):
10         # Create the main window widget.
11         self.main_window = tkinter.Tk()
12
```

```
13         # Create a Button widget. The text 'Click Me!'
14         # should appear on the face of the Button. The
15         # do_something method should be executed when
16         # the user clicks the Button.
17         self.my_button = tkinter.Button(self.main_window,
18                                         text='Click Me!',
19
20
21         command=self.do_something)
22
23         # Pack the Button.
24         self.my_button.pack()
25
26         # Enter the tkinter main loop.
27         tkinter.mainloop()
28
29
30     def do_something(self):
31         # Display an info dialog box.
32         tkinter.messagebox.showinfo('Response',
33                                     'Thanks for clicking the
34                                     button.')
35
36     # Create an instance of the MyGUI class.
37     my_gui = MyGUI()
```

Line 5 imports the `tkinter` module, and line 6 imports the `tkinter.messagebox` module. Line 11 creates the root widget and assigns it to the `main_window` variable.

The statement in lines 17 through 19 creates the `Button` widget. The first argument inside the parentheses is `self.main_window`, which is the parent widget. The `text='Click Me!'` argument specifies that the string 'Click Me!' should appear on the face of the button. The `command='self.do_something'` argument specifies the class's `do_something` method as the callback function. When the user clicks the button, the `do_something` method will execute.

The `do_something` method appears in lines 31 through 33. The method simply calls the `tkinter.messagebox.showinfo` function to display the info box shown in [Figure 13-11](#) . To dismiss the dialog box, the user can click the OK button.

Creating a Quit Button

GUI programs usually have a *Quit button* (or an Exit button) that closes the program when the user clicks it. To create a Quit button in a Python program, you simply create a `Button` widget that calls the root widget's `destroy` method as a callback function. [Program 13-8](#)  demonstrates how to do this. It is a modified version of [Program 13-7](#) , with a second `Button` widget added as shown in [Figure 13-12](#) .

Figure 13-12 The info dialog box displayed by [Program 13-8](#) 



Program 13-8 *(quit_button.py)*

```
1  # This program has a Quit button that calls
2  # the Tk class's destroy method when clicked.
3
4  import tkinter
5  import tkinter.messagebox
6
7  class MyGUI:
8      def __init__(self):
9          # Create the main window widget.
10         self.main_window = tkinter.Tk()
11
12         # Create a Button widget. The text 'Click Me!'
13         # should appear on the face of the Button. The
14         # do_something method should be executed when
15         # the user clicks the Button.
16         self.my_button = tkinter.Button(self.main_window,
17                                         text='Click Me!',
18
19                                         command=self.do_something)
20
21         # Create a Quit button. When this button is
```

```

Clicked
21         # the root widget's destroy method is called.
22         # (The main_window variable references the root
widget,
23         # so the callback function is
self.main_window.destroy.)
24         self.quit_button =
tkinter.Button(self.main_window,
25                 text='Quit',
26
command=self.main_window.destroy)
27
28
29         # Pack the Buttons.
30         self.my_button.pack()
31         self.quit_button.pack()
32
33         # Enter the tkinter main loop.
34         tkinter.mainloop()
35
36         # The do_something method is a callback function
37         # for the Button widget.
38
39         def do_something(self):
40             # Display an info dialog box.
41             tkinter.messagebox.showinfo('Response',
42                 'Thanks for clicking the
button.')
```

```
43
44 # Create an instance of the MyGUI class.
45 my_gui = MyGUI()
```

The statement in lines 24 through 26 creates the Quit button. Notice the `self.main_window.destroy` method is used as the callback function. When the user clicks the button, this method is called and the program ends.

13.6 Getting Input with the Entry Widget

Concept:

An `Entry` widget is a rectangular area that the user can type input into. You use the `Entry` widget's `get` method to retrieve the data that has been typed into the widget.

An `Entry` widget is a rectangular area that the user can type text into. Entry widgets are used to gather input in a GUI program. Typically, a program will have one or more `Entry` widgets in a window, along with a button that the user clicks to submit the data that he or she has typed into the `Entry` widgets. The button's callback function retrieves data from the window's `Entry` widgets and processes it.

You use an `Entry` widget's `get` method to retrieve the data that the user has typed into the widget. The `get` method returns a string, so it will have to be converted to the appropriate data type if the `Entry` widget is used for numeric input.

To demonstrate, we will look at a program that allows the user to enter a distance in kilometers into an `Entry` widget then click a button to see that distance converted to miles. The formula for converting kilometers to miles is:

$$\text{Miles} = \text{Kilometers} \times 0.6214$$

Figure 13-13  shows the window that the program displays. To arrange the widgets in the positions shown in the figure, we will organize them in two frames, as shown in **Figure 13-14** . The label that displays the prompt and the `Entry` widget will be stored in the `top_frame`, and their `pack` methods will be called with the `side='left'` argument. This will cause them to appear horizontally in the frame. The Convert button and the Quit button will be stored in the `bottom_frame`, and their `pack` methods will also be called with the `side='left'` argument.

Figure 13-13 The `kilo_converter` program's window

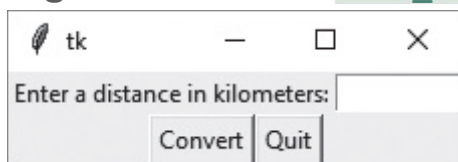
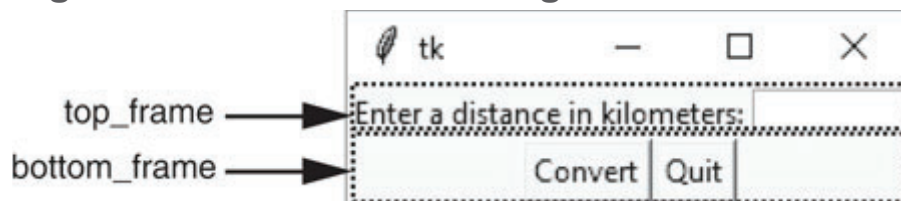
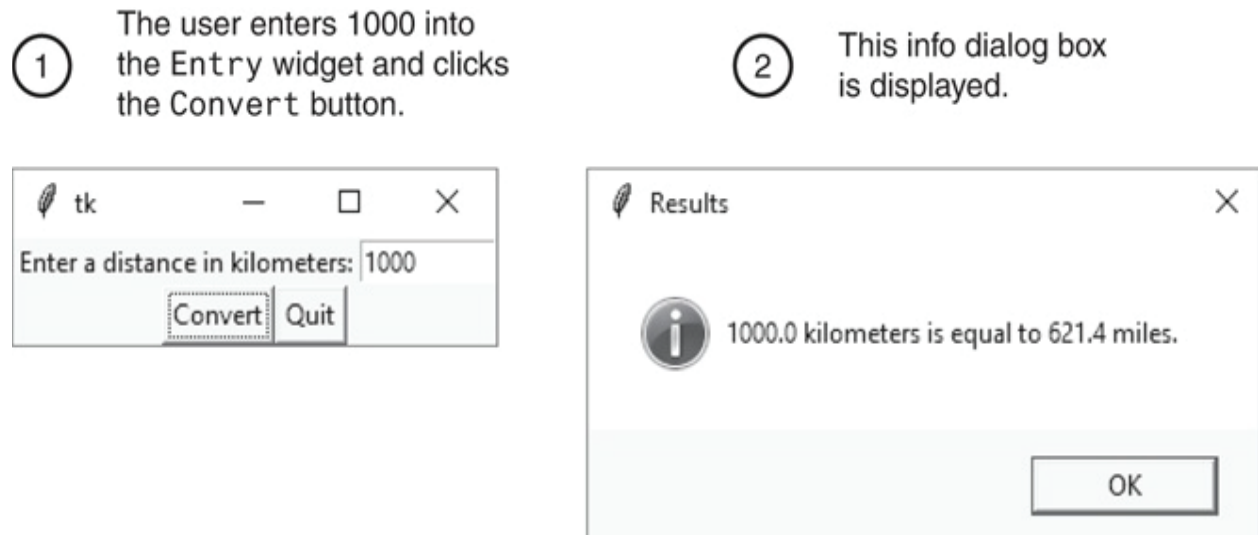


Figure 13-14 The window organized with frames



Program 13-9  shows the code for the program. **Figure 13-15**  shows what happens when the user enters 1000 into the `Entry` widget and clicks the Convert button.

Figure 13-15 The info dialog box



Program 13-9 `(kilo_converter.py)`

```
1  # This program converts distances in kilometers
2  # to miles. The result is displayed in an info
3  # dialog box.
4
5  import tkinter
6  import tkinter.messagebox
7
8  class KiloConverterGUI:
9      def __init__(self):
10
11          # Create the main window.
```

```
12         self.main_window = tkinter.Tk()
13
14         # Create two frames to group widgets.
15         self.top_frame = tkinter.Frame(self.main_window)
16         self.bottom_frame =
tkinter.Frame(self.main_window)
17
18         # Create the widgets for the top frame.
19         self.prompt_label = tkinter.Label(self.top_frame,
20                                           text='Enter a distance in kilometers:')
21         self.kilo_entry = tkinter.Entry(self.top_frame,
22                                         width=10)
23
24         # Pack the top frame's widgets.
25         self.prompt_label.pack(side='left')
26         self.kilo_entry.pack(side='left')
27
28         # Create the button widgets for the bottom frame.
29         self.calc_button =
tkinter.Button(self.bottom_frame,
30              text='Convert',
31
32              command=self.convert)
33
34         self.quit_button =
tkinter.Button(self.bottom_frame,
35              text='Quit',
36
37              command=self.main_window.destroy)
```

[illegible]

```
62
63 # Create an instance of the KiloConverterGUI class.
64 kilo_conv = KiloConverterGUI()
```

The `convert` method, shown in lines 49 through 60 is the Convert button's callback function. The statement in line 52 calls the `kilo_entry` widget's `get` method to retrieve the data that has been typed into the widget. The value is converted to a `float` then assigned to the `kilo` variable. The calculation in line 55 performs the conversion and assigns the results to the `miles` variable. Then, the statement in lines 58 through 61 displays the info dialog box with a message that gives the converted value.

13.7 Using Labels as Output Fields

Concept:

When a `StringVar` object is associated with a `Label` widget, the `Label` widget displays any data that is stored in the `StringVar` object.

Previously, you saw how to use an info dialog box to display output. If you don't want to display a separate dialog box for your program's output, you can use `Label` widgets in the program's main window to dynamically display output. You simply create empty `Label` widgets in your main window, then write code that displays the desired data in those labels when a button is clicked.

The `tkinter` module provides a class named `StringVar` that can be used along with a `Label` widget to display data. First, you create a `StringVar` object. Then, you create a `Label` widget and associate it with the `StringVar` object. From that point on, any value that is then stored in the `StringVar` object will automatically be displayed in the `Label` widget.

Program 13-10  demonstrates how to do this. It is a modified version of the `kilo_converter` program that you saw in **Program 13-9** . Instead of popping up an info dialog box, this version of the program displays the number of miles in a label in the main window.

Program 13-10 `(kilo_converter2.py)`

```
1  # This program converts distances in kilometers
2  # to miles. The result is displayed in a label
3  # on the main window.
4
5  import tkinter
6
7  class KiloConverterGUI:
8      def __init__(self):
9
10         # Create the main window.
11         self.main_window = tkinter.Tk()
12
13         # Create three frames to group widgets.
14         self.top_frame = tkinter.Frame()
15         self.mid_frame = tkinter.Frame()
16         self.bottom_frame = tkinter.Frame()
17
18         # Create the widgets for the top frame.
19         self.prompt_label = tkinter.Label(self.top_frame,
20                                           text='Enter a distance in kilometers:')
21         self.kilo_entry = tkinter.Entry(self.top_frame,
```



```
22                                     width=10)
23
24     # Pack the top frame's widgets.
25     self.prompt_label.pack(side='left')
26     self.kilo_entry.pack(side='left')
27
28     # Create the widgets for the middle frame.
29     self.descr_label = tkinter.Label(self.mid_frame,
30                                     text='Converted to
miles:')
31
32     # We need a StringVar object to associate with
33     # an output label. Use the object's set method
34     # to store a string of blank characters.
35     self.value = tkinter.StringVar()
36
37     # Create a label and associate it with the
38     # StringVar object. Any value stored in the
39     # StringVar object will automatically be displayed
40     # in the label.
41     self.miles_label = tkinter.Label(self.mid_frame,
42                                     textvariable=self.value)
43
44     # Pack the middle frame's widgets.
45     self.descr_label.pack(side='left')
46     self.miles_label.pack(side='left')
47
48     # Create the button widgets for the bottom frame.
```

```
49         self.calc_button =
tkinter.Button(self.bottom_frame,
50                                     text='Convert',
51
command=self.convert)
52         self.quit_button =
tkinter.Button(self.bottom_frame,
53                                     text='Quit',
54
command=self.main_window.destroy)
55
56         # Pack the buttons.
57         self.calc_button.pack(side='left')
58         self.quit_button.pack(side='left')
59
60         # Pack the frames.
61         self.top_frame.pack()
62         self.mid_frame.pack()
63         self.bottom_frame.pack()
64
65         # Enter the tkinter main loop.
66         tkinter.mainloop()
67
68         # The convert method is a callback function for
69         # the Calculate button.
70
71         def convert(self):
72             # Get the value entered by the user into the
```

```

73         # kilo_entry widget.
74         kilo = float(self.kilo_entry.get())
75
76         # Convert kilometers to miles.
77         miles = kilo * 0.6214
78
79         # Convert miles to a string and store it
80         # in the StringVar object. This will automatically
81         # update the miles_label widget.
82         self.value.set(miles)
83
84     # Create an instance of the KiloConverterGUI class.
85     kilo_conv = KiloConverterGUI()

```

When this program runs, it displays the window shown in **Figure 13-16**. **Figure 13-17** shows what happens when the user enters 1000 for the kilometers and clicks the Convert button. The number of miles is displayed in a label in the main window.

Figure 13-16 The window initially displayed

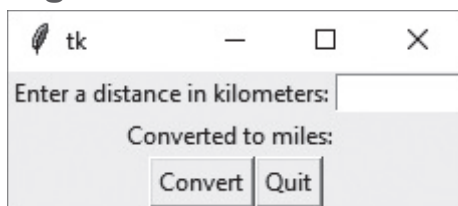
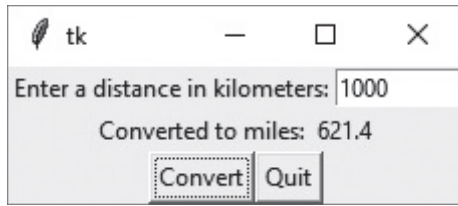


Figure 13-17 The window showing 1000 kilometers converted to miles

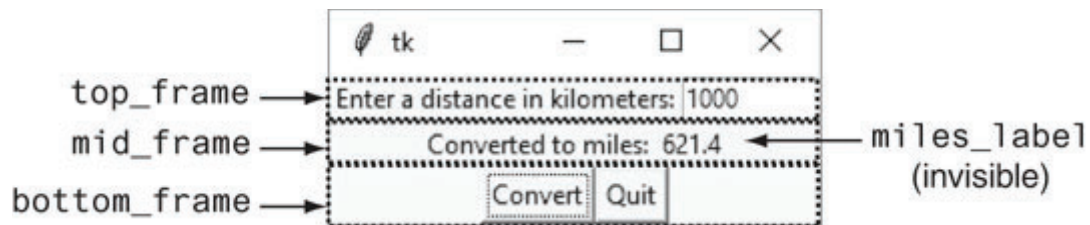


Let's look at the code. Lines 14 through 16 create three frames: `top_frame`, `mid_frame`, and `bottom_frame`. Lines 19 through 26 create the widgets for the top frame and call their `pack` method.

Lines 29 through 30 create the Label widget with the text `'Converted to miles:'` that you see on the main window in [Figure 13-16](#). Then, line 35 creates a `StringVar` object and assigns it to the `value` variable. Line 41 creates a `Label` widget named `miles_label` that we will use to display the number of miles. Notice in line 42, we use the argument `textvariable=self.value`. This creates an association between the `Label` widget and the `StringVar` object that is referenced by the `value` variable. Any value that we store in the `StringVar` object will be displayed in the label.

Lines 45 and 46 pack the two `Label` widgets that are in the `mid_frame`. Lines 49 through 58 create the `Button` widgets and `pack` them. Lines 61 through 63 pack the `Frame` objects. [Figure 13-18](#) shows how the various widgets in this window are organized in the three frames.

Figure 13-18 Layout of the `kilo_converter2` program's main window



The `convert` method, shown in lines 71 through 82 is the Convert button's callback function. The statement in line 74 calls the `kilo_entry` widget's `get` method to retrieve the data that has been typed into the widget. The value is converted to a `float` then assigned to the `kilo` variable. The calculation in line 77 performs the conversion and assigns the results to the `miles` variable. Then the statement in line 82 calls the `StringVar` object's `set` method, passing `miles` as an argument. This stores the value referenced by `miles` in the `StringVar` object, and also causes it to be displayed in the `miles_label` widget.



In the

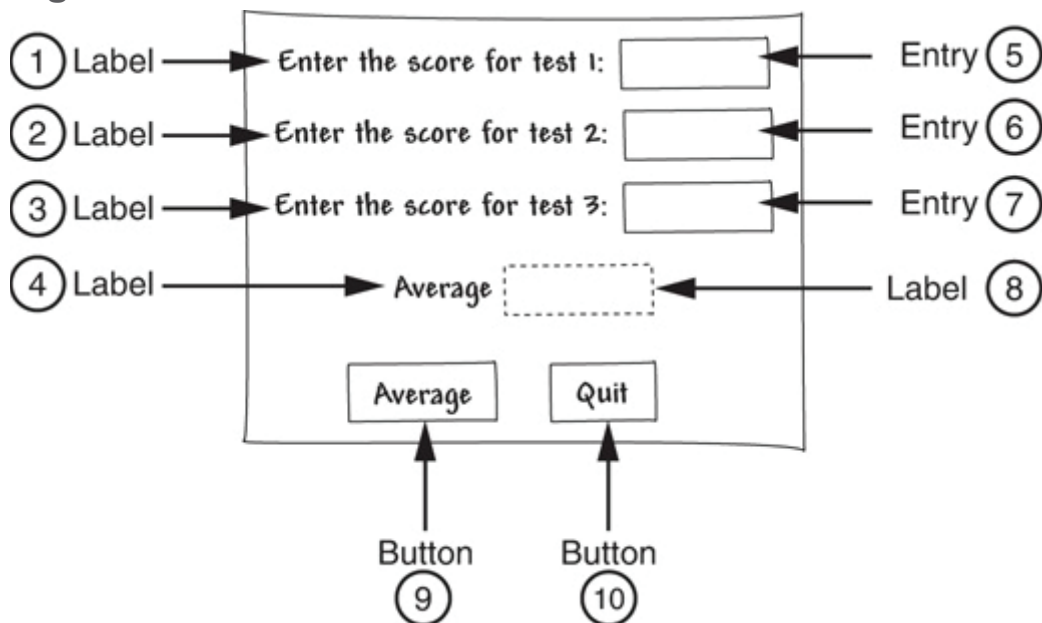
Spotlight: Creating a GUI Program

Kathryn teaches a science class. In [Chapter 3](#), we stepped through the development of a program that her students can use to calculate the average of three test scores. The program prompts the student to enter each score, then it displays the average. She has asked you to design a GUI program that performs a similar operation. She would like the program to have three `Entry` widgets into which the test scores can be

entered, and a button that causes the average to be displayed when clicked.

Before we begin writing code, it will be helpful if we draw a sketch of the program's window, as shown in [Figure 13-19](#). The sketch also shows the type of each widget. (The numbers that appear in the sketch will help us when we make a list of all the widgets.)

Figure 13-19 A sketch of the window



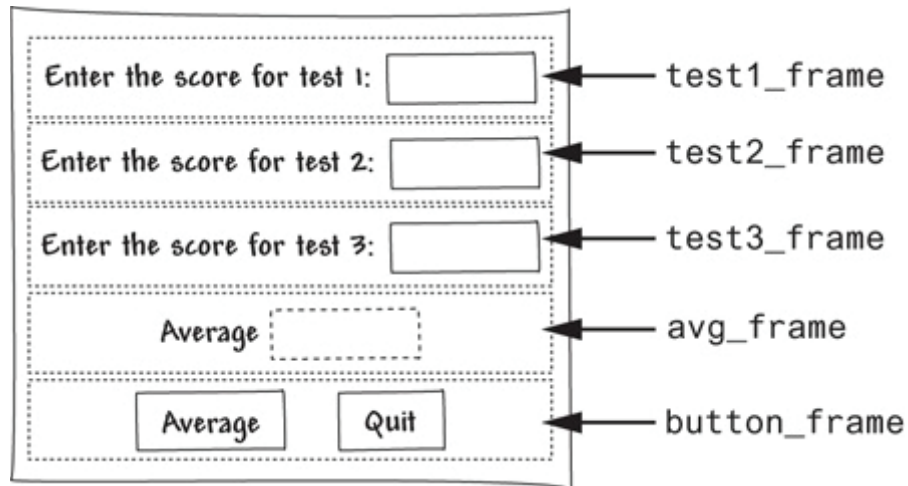
By examining the sketch, we can make a list of the widgets that we need. As we make the list, we will include a brief description of each widget, and a name that we will assign to each widget when we construct it.

Widget Number in Figure 13-19	Widget Type	Description	Name

1	Label	Instructs the user to enter the score for test 1.	<code>test1_label</code>
2	Label	Instructs the user to enter the score for test 2.	<code>test2_label</code>
3	Label	Instructs the user to enter the score for test 3.	<code>test3_label</code>
4	Label	Identifies the average, which will be displayed next to this label.	<code>result_label</code>
5	Entry	This is where the user will enter the score for test 1.	<code>test1_entry</code>
6	Entry	This is where the user will enter the score for test 2.	<code>test2_entry</code>
7	Entry	This is where the user will enter the score for test 3.	<code>test3_entry</code>
8	Label	The program will display the average test score in this label.	<code>avg_label</code>
9	Button	When this button is clicked, the program will calculate the average test score and display it in the <code>averageLabel</code> component.	<code>calc_button</code>
10	Button	When this button is clicked the program will end.	<code>quit_button</code>

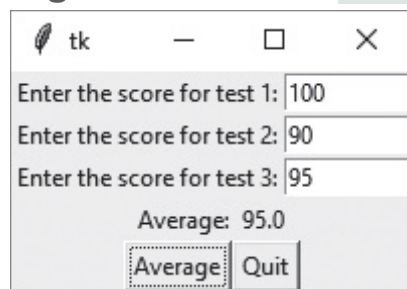
We can see from the sketch that we have five rows of widgets in the window. To organize them, we will also create five `Frame` objects. **Figure 13-20** shows how we will position the widgets inside the five `Frame` objects.

Figure 13-20 Using Frames to organize the widgets



Program 13-11 shows the code for the program, and **Figure 13-21** shows the program's window with data entered by the user.

Figure 13-21 The `test_averages` program window



Program 13-11

(test_averages.py)

```
1  # This program uses a GUI to get three test
2  # scores and display their average.
3
4  import tkinter
5
6  class TestAvg:
7      def __init__(self):
8          # Create the main window.
9          self.main_window = tkinter.Tk()
10
11         # Create the five frames.
12         self.test1_frame =
tkinter.Frame(self.main_window)
13         self.test2_frame =
tkinter.Frame(self.main_window)
14         self.test3_frame =
tkinter.Frame(self.main_window)
15         self.avg_frame =
tkinter.Frame(self.main_window)
16         self.button_frame =
tkinter.Frame(self.main_window)
17
18         # Create and pack the widgets for test 1.
19         self.test1_label =
tkinter.Label(self.test1_frame,
```

```
20
text='Enter the score for test 1:')
21     self.test1_entry =
tkinter.Entry(self.test1_frame,
22                                     width=10)
23     self.test1_label.pack(side='left')
24     self.test1_entry.pack(side='left')
25
26     # Create and pack the widgets for test 2.
27     self.test2_label =
tkinter.Label(self.test2_frame,
28
text='Enter the score for test 2:')
29     self.test2_entry =
tkinter.Entry(self.test2_frame,
30                                     width=10)
31     self.test2_label.pack(side='left')
32     self.test2_entry.pack(side='left')
33
34     # Create and pack the widgets for test 3.
35     self.test3_label =
tkinter.Label(self.test3_frame,
36
text='Enter the score for test 3:')
37     self.test3_entry =
tkinter.Entry(self.test3_frame,
38                                     width=10)
39     self.test3_label.pack(side='left')
```

```
40         self.test3_entry.pack(side='left')
41
42         # Create and pack the widgets for the
average.
43         self.result_label =
tkinter.Label(self.avg_frame,
44
text='Average:')
45         self.avg = tkinter.StringVar() # To update
avg_label
46         self.avg_label =
tkinter.Label(self.avg_frame,
47
textvariable=self.avg)
48         self.result_label.pack(side='left')
49         self.avg_label.pack(side='left')
50
51         # Create and pack the button widgets.
52         self.calc_button =
tkinter.Button(self.button_frame,
53
text='Average',
54
command=self.calc_avg)
55         self.quit_button =
tkinter.Button(self.button_frame,
56
text='Quit',
```

```
57
command=self.main_window.destroy)
58         self.calc_button.pack(side='left')
59         self.quit_button.pack(side='left')
60
61         # Pack the frames.
62         self.test1_frame.pack()
63         self.test2_frame.pack()
64         self.test3_frame.pack()
65         self.avg_frame.pack()
66         self.button_frame.pack()
67
68         # Start the main loop.
69         tkinter.mainloop()
70
71         # The calc_avg method is the callback function
for
72         # the calc_button widget.
73
74         def calc_avg(self):
75             # Get the three test scores and store them
76             # in variables.
77             self.test1 = float(self.test1_entry.get())
78             self.test2 = float(self.test2_entry.get())
79             self.test3 = float(self.test3_entry.get())
80
81             # Calculate the average.
82             self.average = (self.test1 + self.test2 +
```

```
83         self.test3) / 3.0
84
85         # Update the avg_label widget by storing
86         # the value of self.average in the
StringVar
87         # object referenced by avg.
88         self.avg.set(self.average)
89
90     # Create an instance of the TestAvg class.
91     test_avg = TestAvg()
```



Checkpoint

13.11 How do you retrieve data from an `Entry` widget?

13.12 When you retrieve a value from an `Entry` widget, of what data type is it?

13.13 What module is the `StringVar` class in?

13.14 What can you accomplish by associating a `StringVar` object with a `Label` widget?

13.8 Radio Buttons and Check Buttons

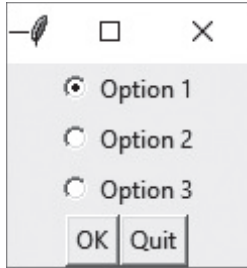
Concept:

Radio buttons normally appear in groups of two or more, and allow the user to select one of several possible options. Check buttons, which may appear alone or in groups, allow the user to make yes/no or on/off selections.

Radio Buttons

Radio buttons are useful when you want the user to select one choice from several possible options. **Figure 13-22**  shows a window containing a group of radio buttons. A radio button may be selected or deselected. Each radio button has a small circle that appears filled in when the radio button is selected, and appears empty when the radio button is deselected.

Figure 13-22 A group of radio buttons



You use the `tkinter` module's `Radiobutton` class to create `Radiobutton` widgets. Only one of the `Radiobutton` widgets in a container, such as a `Frame`, may be selected at any time. Clicking a `Radiobutton` selects it and automatically deselects any other `Radiobutton` in the same container. Because only one `Radiobutton` in a container can be selected at any given time, they are said to be *mutually exclusive*.



Note:

The name “radio button” refers to the old car radios that had push buttons for selecting stations. Only one of the buttons could be pushed in at a time. When you pushed a button in, it automatically popped out any other button that was pushed in.

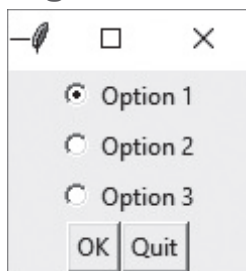
The `tkinter` module provides a class named `IntVar` that can be used along with `Radiobutton` widgets. When you create a group of `Radiobuttons`, you associate them all with the same `IntVar` object. You also assign a unique integer value to each `Radiobutton` widget. When

one of the `Radiobutton` widgets is selected, it stores its unique integer value in the `IntVar` object.

Program 13-12  demonstrates how to create and use `Radiobuttons`.

Figure 13-23  shows the window that the program displays. When the user clicks the OK button, an info dialog box appears indicating which of the `Radiobuttons` is selected.

Figure 13-23 Window displayed by **Program 13-12** 



Program 13-12 (`radiobutton_demo.py`)

```
1  # This program demonstrates a group of Radiobutton
   widgets.
2  import tkinter
3  import tkinter.messagebox
4
5  class MyGUI:
6      def __init__(self):
7          # Create the main window.
8          self.main_window = tkinter.Tk()
9
10         # Create two frames. One for the Radiobuttons
```



```
11         # and another for the regular Button widgets.
12         self.top_frame = tkinter.Frame(self.main_window)
13         self.bottom_frame =
tkinter.Frame(self.main_window)
14
15         # Create an IntVar object to use with
16         # the Radiobuttons.
17         self.radio_var = tkinter.IntVar()
18
19         # Set the intVar object to 1.
20         self.radio_var.set(1)
21
22         # Create the Radiobutton widgets in the top_frame.
23         self.rb1 = tkinter.Radiobutton(self.top_frame,
24                                         text='Option 1',
25                                         variable=self.radio_var,
26                                         value=1)
27         self.rb2 = tkinter.Radiobutton(self.top_frame,
28                                         text='Option 2',
29                                         variable=self.radio_var,
30                                         value=2)
31         self.rb3 = tkinter.Radiobutton(self.top_frame,
32                                         text='Option 3',
33                                         variable=self.radio_var,
34                                         value=3)
```

```
35
36     # Pack the Radiobuttons.
37     self.rb1.pack()
38     self.rb2.pack()
39     self.rb3.pack()
40
41     # Create an OK button and a Quit button.
42     self.ok_button = tkinter.Button(self.bottom_frame,
43                                     text='OK',
44
45                                     command=self.show_choice)
46     self.quit_button =
47     tkinter.Button(self.bottom_frame,
48                     text='Quit',
49
50                     command=self.main_window.destroy)
51
52     # Pack the Buttons.
53     self.ok_button.pack(side='left')
54     self.quit_button.pack(side='left')
55
56     # Pack the frames.
57     self.top_frame.pack()
58     self.bottom_frame.pack()
59
60     # Start the mainloop.
61     tkinter.mainloop()
62
```

```

60     # The show_choice method is the callback function for
the
61     # OK button.
62     def show_choice(self):
63         tkinter.messagebox.showinfo('Selection', 'You
selected option ' +
64
str(self.radio_var.get()))
65
66     # Create an instance of the MyGUI class.
67     my_gui = MyGUI()

```

Line 17 creates an `IntVar` object named `radio_var`. Line 20 calls the `radio_var` object's `set` method to store the integer value 1 in the object. (You will see the significance of this in a moment.)

Lines 23 through 26 create the first `Radiobutton` widget. The argument `variable = self.radio_var` (in line 25) associates the `Radiobutton` with the `radio_var` object. The argument `value=1` (in line 26) assigns the integer 1 to this `Radiobutton`. As a result, any time this `Radiobutton` is selected, the value 1 will be stored in the `radio_var` object.

Lines 27 through 30 create the second `Radiobutton` widget. Notice this `Radiobutton` is also associated with the `radio_var` object. The argument `value=2` (in line 30) assigns the integer 2 to this `Radiobutton`.

As a result, any time this `Radiobutton` is selected, the value 2 will be stored in the `radio_var` object.

Lines 31 through 34 create the third `Radiobutton` widget. This `Radiobutton` is also associated with the `radio_var` object. The argument `value=3` (in line 34) assigns the integer 3 to this `Radiobutton`. As a result, any time this `Radiobutton` is selected, the value 3 will be stored in the `radio_var` object.

The `show_choice` method in lines 62 through 64 is the callback function for the OK button. When the method executes, it calls the `radio_var` object's `get` method to retrieve the value stored in the object. The value is displayed in an info dialog box.

Did you notice when the program runs, the first `Radiobutton` is initially selected? This is because we set the `radio_var` object to the value 1 in line 20. Not only can the `radio_var` object be used to determine which `Radiobutton` was selected, but it can also be used to select a specific `Radiobutton`. When we store a particular `Radiobutton`'s value in the `radio_var` object, that `Radiobutton` will become selected.

Using Callback Functions with Radiobuttons

Program 13-12  waits for the user to click the OK button before it determines which `Radiobutton` was selected. If you prefer, you can also specify a callback function with `Radiobutton` widgets. Here is an example:

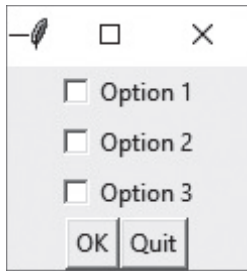
```
self.rb1 = tkinter.Radiobutton(self.top_frame,  
                                text='Option 1',  
                                variable=self.radio_var,  
                                value=1,  
                                command=self.my_method)
```

This code uses the argument `command=self.my_method` to specify that `my_method` is the callback function. The method `my_method` will be executed immediately when the `Radiobutton` is selected.

Check Buttons

A *check button* appears as a small box with a label appearing next to it. The window shown in **Figure 13-24**  has three check buttons.

Figure 13-24 A group of check buttons



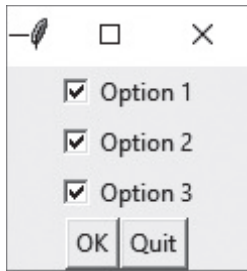
Like radio buttons, check buttons may be selected or deselected. When a check button is selected, a small check mark appears inside its box. Although check buttons are often displayed in groups, they are not used to make mutually exclusive selections. Instead, the user is allowed to select any or all of the check buttons that are displayed in a group.

You use the `tkinter` module's `Checkbutton` class to create `Checkbutton` widgets. As with `Radiobuttons`, you can use an `IntVar` object along with a `Checkbutton` widget. Unlike `Radiobuttons`, however, you associate a different `IntVar` object with each `Checkbutton`. When a `Checkbutton` is selected, its associated `IntVar` object will hold the value 1. When a `Checkbutton` is not selected, its associated `IntVar` object will hold the value 0.

Program 13-13  demonstrates how to create and use `Checkbuttons`.

Figure 13-25  shows the window that the program displays. When the user clicks the OK button, an info dialog box appears indicating which of the `Checkbuttons` is selected.

Figure 13-25 Window displayed by **Program 13-13** 



Program 13-13 (*checkboxbutton_demo.py*)

```
1  # This program demonstrates a group of Checkbutton
widgets.
2  import tkinter
3  import tkinter.messagebox
4
5  class MyGUI:
6      def __init__(self):
7          # Create the main window.
8          self.main_window = tkinter.Tk()
9
10         # Create two frames. One for the checkbuttons
11         # and another for the regular Button widgets.
12         self.top_frame = tkinter.Frame(self.main_window)
13         self.bottom_frame =
tkinter.Frame(self.main_window)
14
15         # Create three IntVar objects to use with
16         # the Checkbuttons.
17         self.cb_var1 = tkinter.IntVar()
18         self.cb_var2 = tkinter.IntVar()
```

[illegible]


```
46         self.quit_button =
tkinter.Button(self.bottom_frame,
47                                     text='Quit',
48
command=self.main_window.destroy)
49
50         # Pack the Buttons.
51         self.ok_button.pack(side='left')
52         self.quit_button.pack(side='left')
53
54         # Pack the frames.
55         self.top_frame.pack()
56         self.bottom_frame.pack()
57
58         # Start the mainloop.
59         tkinter.mainloop()
60
61         # The show_choice method is the callback function for
the
62         # OK button.
63
64         def show_choice(self):
65             # Create a message string.
66             self.message = 'You selected:\n'
67
68             # Determine which Checkbuttons are selected and
69             # build the message string accordingly.
70             if self.cb_var1.get() == 1:
```

```

71         self.message = self.message + '1\n'
72         if self.cb_var2.get() == 1:
73             self.message = self.message + '2\n'
74             if self.cb_var3.get() == 1:
75                 self.message = self.message + '3\n'
76
77         # Display the message in an info dialog box.
78         tkinter.messagebox.showinfo('Selection',
self.message)
79
80     # Create an instance of the MyGUI class.
81     my_gui = MyGUI()

```



Checkpoint

13.15 You want the user to be able to select only one item from a group of items. Which type of component would you use for the items, radio buttons or check buttons?

13.16 You want the user to be able to select any number of items from a group of items. Which type of component would you use for the items, radio buttons or check buttons?

13.17 How can you use an `IntVar` object to determine which `Radiobutton` has been selected in a group of `Radiobuttons`?

13.18 How can you use an `IntVar` object to determine whether a `Checkbutton` has been selected?

13.9 Drawing Shapes with the Canvas Widget

Concept:

The `Canvas` widget provides methods for drawing simple shapes, such as lines, rectangles, ovals, polygons, and so on.

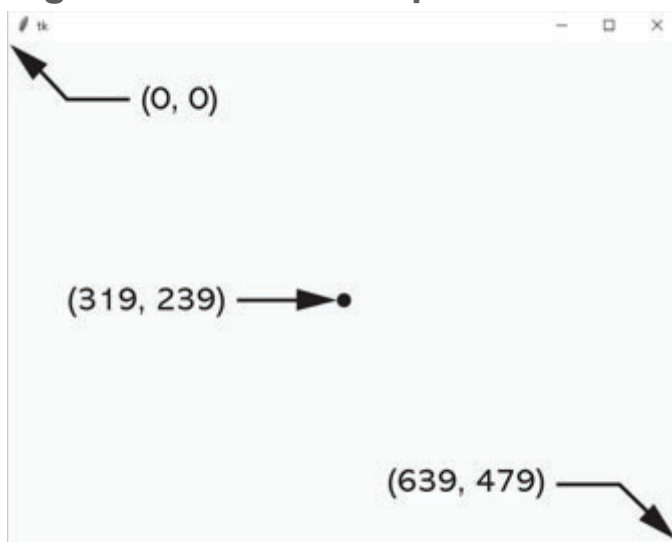
The `Canvas` widget is a blank, rectangular area that allows you to draw simple 2D shapes. In this section, we will discuss `Canvas` methods for drawing lines, rectangles, ovals, arcs, polygons, and text. Before we examine how to draw these shapes, however, we must discuss the screen coordinate system. You use the `Canvas` widget's *screen coordinate system* to specify the location of your graphics.

The `Canvas` Widget's Screen Coordinate System

The images that are displayed on a computer screen are made up of tiny dots called *pixels*. The screen coordinate system is used to identify the position of each pixel in an application's window. Each pixel has an *X* coordinate and a *Y* coordinate. The *X* coordinate identifies the pixel's horizontal position, and the *Y* coordinate identifies its vertical position. The coordinates are usually written in the form (*X*, *Y*).

In the Canvas widget's screen coordinate system, the coordinates of the pixel in the upper-left corner of the screen are (0, 0). This means that its *X* coordinate is 0 and its *Y* coordinate is 0. The *X* coordinates increase from left to right, and the *Y* coordinates increase from top to bottom. In a window that is 640 pixels wide by 480 pixels high, the coordinates of the pixel at the bottom right corner of the window are (639, 479). In the same window, the coordinates of the pixel in the center of the window are (319, 239). **Figure 13-26**  shows the coordinates of various pixels in the window.

Figure 13-26 Various pixel locations in a 640 by 480 window





Note:

The `Canvas` widget's screen coordinate system differs from the Cartesian coordinate system that is used by the turtle graphics library. Here are the differences:

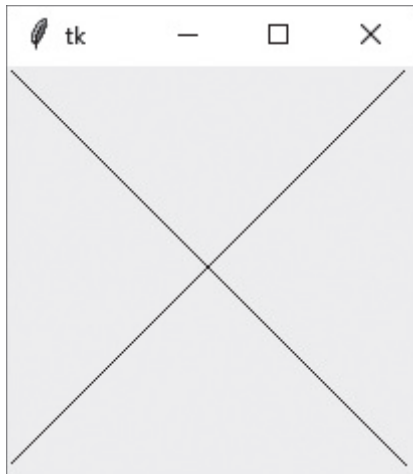
- With the `Canvas` widget, the point (0, 0) is in the upper left corner of the window. In turtle graphics, the point (0, 0) is in the center of the window.
- With the `Canvas` widget, the Y coordinates increase as you move down the screen. In turtle graphics, the Y coordinates decrease as you move down the screen.

The `Canvas` widget has numerous methods for drawing graphical shapes on the surface of the widget. The methods that we will discuss are:

- `create_line`
- `create_rectangle`
- `create_oval`
- `create_arc`
- `create_polygon`
- `create_text`

Before discussing the details of these methods, let's look at **Program 13-14** . It is a simple program that uses a `Canvas` widget to draw lines. **Figure 13-27**  shows the window that the program displays.

Figure 13-27 Window displayed by Program 13-14 



Program 13-14 (`draw_line.py`)

```
1  # This program demonstrates the Canvas widget.
2  import tkinter
3
4  class MyGUI:
5      def __init__(self):
6          # Create the main window.
7          self.main_window = tkinter.Tk()
8
9          # Create the Canvas widget.
10         self.canvas = tkinter.Canvas(self.main_window,
width=200,height=200)
11
```

```
12         # Draw two lines.
13         self.canvas.create_line(0, 0, 199, 199)
14         self.canvas.create_line(199, 0, 0, 199)
15
16         # Pack the canvas.
17         self.canvas.pack()
18
19         # Start the mainloop.
20         tkinter.mainloop()
21
22     # Create an instance of the MyGUI class.
23     my_gui = MyGUI()
```

Let's take a closer look at the program. Line 10 creates the `Canvas` widget. The first argument inside the parentheses is a reference to `self.main_window`, which is the parent container to which we are adding the widget. The arguments `width=200` and `height=200` specify the size of the `Canvas` widget.

Line 13 calls the `Canvas` widget's `create_line` method, to draw a line. The first and second arguments are the (X, Y) coordinates of the line's starting point. The third and fourth arguments are the (X, Y) coordinates of the line's ending point. So, this statement draws a line on the `Canvas`, from (0, 0) to (199, 199).

Line 14 also calls the `Canvas` widget's `create_line` method, to draw a second line. This statement draws a line on the `Canvas`, from (199, 0)

to (0, 199).

Line 17 calls the `Canvas` widget's `pack` method, which makes the widget visible. Line 20 executes the `tkinter` module's `mainloop` function.

Drawing Lines: The `create_line` Method

The `create_line` method draws a line between two or more points on the Canvas. Here is the general format of how you call the method to draw a line between two points:

```
canvas_name.create_line(x1, y1, x2, y2, options...)
```

The arguments `x1` and `y1` are the (X, Y) coordinates of the line's starting point. The arguments `x2` and `y2` are the (X, Y) coordinates of the line's ending point. In the general format, `options ...` indicates several optional keyword arguments that you may pass to the method. We will examine some of those in [Table 13-2](#) .

Table 13-2 Some of the optional arguments to the `create_line` method

--	--

Argument	Description
<code>arrow=value</code>	By default, lines do not have arrowheads, but this argument causes the line to have an arrowhead at one or both ends. Specify <code>arrow=tk.FIRST</code> to draw an arrowhead at the beginning of the line, <code>arrow=tk.LAST</code> to draw an arrowhead at the end of the line, or <code>arrow=tk.BOTH</code> to draw arrowheads at both ends of the line.
<code>dash=value</code>	This argument causes the line to be a dashed line. The value is a tuple, consisting of integers, that specifies a pattern. The first integer specifies the number of pixels to draw, the second integer specifies the number of pixels to skip, and so forth. For example, the argument <code>dash=(5, 2)</code> will draw 5 pixels, skip 2 pixels, and repeat until the end of the line is reached.
<code>fill=value</code>	Specifies the color of the line. The argument's value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> , <code>'blue'</code> , <code>'yellow'</code> , and <code>'cyan'</code> . (If you omit the <code>fill</code> argument, the default color is black.)
<code>smooth=value</code>	By default, the <code>smooth</code> argument is set to <code>False</code> , which makes the method draw straight lines connecting the specified points. If you specify <code>smooth=True</code> , the lines are drawn as curved splines.
<code>width=value</code>	Specifies the width of the line, in pixels. For example, the argument <code>width=5</code> causes the line to be 5 pixels wide. By default, lines are 1 pixel wide.

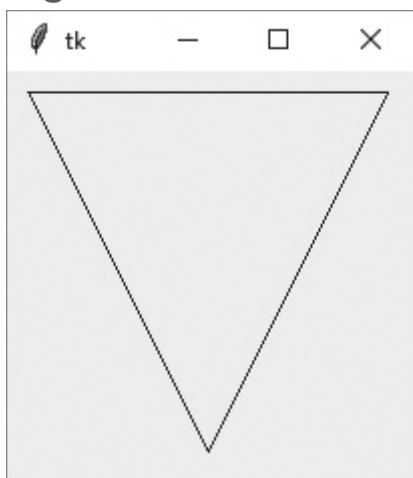
You saw examples of the `create_line` method being called in [Program 13-14](#). Recall that line 13 in the program draws a line from

(0, 0) to (199, 199):

```
self.canvas.create_line(0, 0, 199, 199)
```

You can pass multiple sets of coordinates as arguments. The `create_line` method will draw lines connecting the points. **Program 13-15** demonstrates this. The statement in line 13 draws a line connecting the points (10, 10), (189, 10) (100, 189), and (10, 10). **Figure 13-28** shows the window that the program displays.

Figure 13-28 Window displayed by **Program 13-15**



Program 13-15 (`draw_multi_lines.py`)

```
1  # This program connects multiple points with a line.
2  import tkinter
3
4  class MyGUI:
```

```

5     def __init__(self):
6         # Create the main window.
7         self.main_window = tkinter.Tk()
8
9         # Create the Canvas widget.
10        self.canvas = tkinter.Canvas(self.main_window,
width=200, height=200)
11
12        # Draw a line connecting multiple points.
13        self.canvas.create_line(10, 10, 189, 10, 100, 189,
10, 10)
14
15        # Pack the canvas.
16        self.canvas.pack()
17
18        # Start the mainloop.
19        tkinter.mainloop()
20
21    # Create an instance of the MyGUI class.
22    my_gui = MyGUI()

```

Alternatively, you can pass a list or a tuple containing the coordinates as an argument. For example, in [Program 13-15](#), we could replace line 13 with the following code and get the same results:

```

points = [10, 10, 189, 10, 100, 189, 10, 10]
self.canvas.create_line(points)

```

There are several optional keyword arguments that you can pass to the `create_line` method. [Table 13-2](#) lists some of the more commonly used ones.

Drawing Rectangles: The `create_rectangle` Method

The `create_rectangle` method draws a rectangle on the Canvas. Here is the general format of how you call the method:

```
canvas_name.create_rectangle(x1, y1, x2, y2, options...)
```

The arguments `x1` and `y1` are the (X, Y) coordinates of the rectangle's upper-left corner. The arguments `x2` and `y2` are the (X, Y) coordinates of the rectangle's lower-right corner. In the general format, `options...` indicates several optional keyword arguments that you may pass to the method. We will examine some of those in [Table 13-3](#).

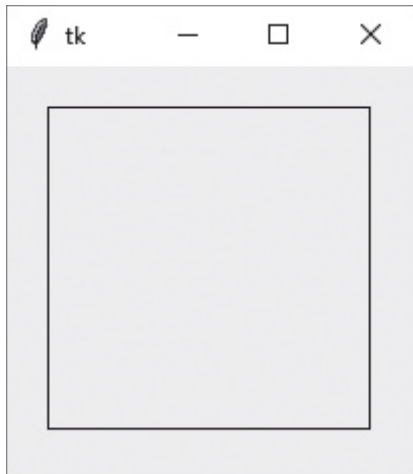
Table 13-3 Some of the optional arguments to the `create_rectangle` method

Argument	Description
<code>dash=value</code>	This argument causes the outline of the rectangle to be a dashed

	line. The value is a tuple, consisting of integers, that specifies a pattern. The first integer specifies the number of pixels to draw, the second integer specifies the number of pixels to skip, and so forth. For example, the argument <code>dash=(5, 2)</code> will draw 5 pixels, skip 2 pixels, and repeat until the end of the line is reached.
<code>fill=value</code>	Specifies a color with which to fill the rectangle. The argument's value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D shows the complete list. Some of the more common colors are <code>'red'</code>, <code>'green'</code>, <code>'blue'</code>, <code>'yellow'</code>, and <code>'cyan'</code>. (If you omit the <code>fill</code> argument, the rectangle will not be filled.)
<code>outline=value</code>	Specifies the color of the rectangle's outline. The argument's value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D shows the complete list. Some of the more common colors are <code>'red'</code>, <code>'green'</code>, <code>'blue'</code>, <code>'yellow'</code>, and <code>'cyan'</code>. (If you omit the <code>outline</code> argument, the default color is black.)
<code>width=value</code>	Specifies the width of the rectangle's outline, in pixels. For example, the argument <code>width=5</code> causes the line to be 5 pixels wide. By default, lines are 1 pixel wide.

Program 13-16 demonstrates the `create_rectangle` method, in line 13. The rectangle's upper-left corner is at (20, 20), and its lower-right corner is at (180, 180). **Figure 13-29** shows the window that the program displays.

Figure 13-29 Window displayed by **Program 13-16**



Program 13-16 (draw_square.py)

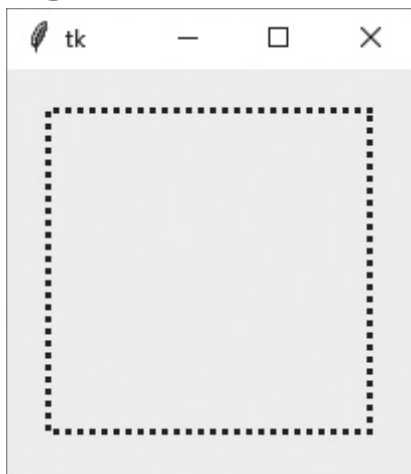
```
1  # This program draws a rectangle on a Canvas.
2  import tkinter
3
4  class MyGUI:
5      def __init__(self):
6          # Create the main window.
7          self.main_window = tkinter.Tk()
8
9          # Create the Canvas widget.
10         self.canvas = tkinter.Canvas(self.main_window,
width=200, height=200)
11
12         # Draw a rectangle.
13         self.canvas.create_rectangle(20, 20, 180, 180)
14
15         # Pack the canvas.
16         self.canvas.pack()
```

```
17
18     # Start the mainloop.
19     tkinter.mainloop()
20
21     # Create an instance of the MyGUI class.
22     my_gui = MyGUI()
```

There are several optional keyword arguments that you can pass to the `create_rectangle` method. [Table 13-3](#) lists some of the more commonly used ones.

For example, if we modify line 13 in [Program 13-16](#) as follows, the program will draw a rectangle with a dashed, 3-pixel wide outline. The program's output would appear as shown in [Figure 13-30](#).

Figure 13-30 Dashed, 3-pixel wide outline



```
self.canvas.create_rectangle(20, 20, 180, 180, dash=(5, 2),
width=3)
```

Drawing Ovals: The `create_oval` Method

The `create_oval` method draws an oval, or an ellipse. Here is the general format of how you call the method:

```
canvas_name.create_oval(x1, y1, x2, y2, options...)
```

The method draws an oval that fits just inside a bounding rectangle defined by the coordinates that are passed as arguments. (`x1`, `y1`) are the coordinates of the rectangle's upper-left corner, and (`x2`, `y2`) are the coordinates of the rectangle's lower-right corner. This is illustrated in [Figure 13-31](#) . In the general format, `options ...` indicates several optional keyword arguments that you may pass to the method. We will examine some of those in [Table 13-4](#) .

Figure 13-31 An oval's bounding rectangle

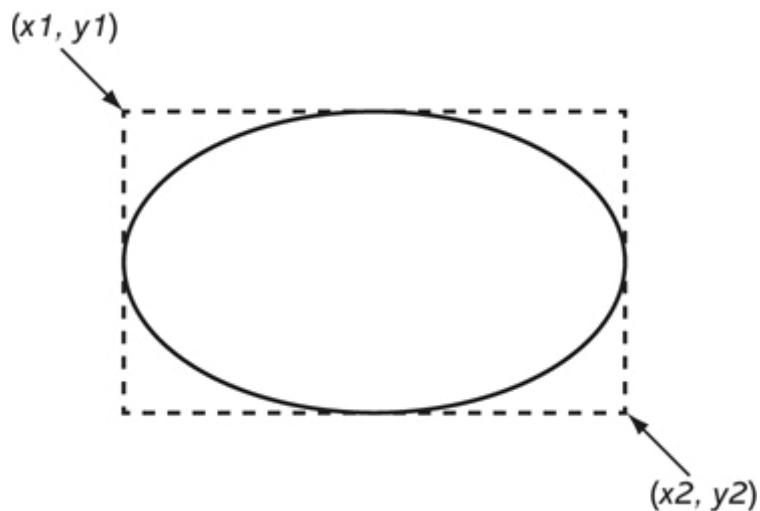


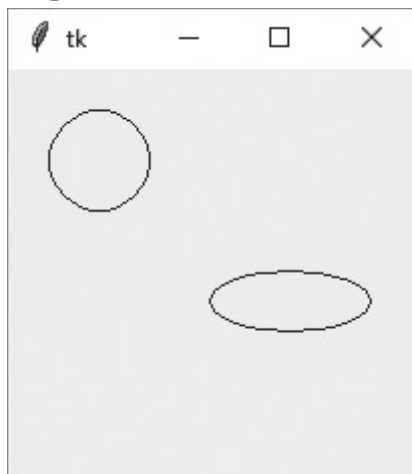
Table 13-4 Some of the optional arguments to the `create_oval` method

Argument	Description
<code>dash=value</code>	This argument causes the outline of the oval to be a dashed line. The value is a tuple, consisting of integers, that specifies a pattern. The first integer specifies the number of pixels to draw, the second integer specifies the number of pixels to skip, and so forth. For example, the argument <code>dash=(5, 2)</code> will draw 5 pixels, skip 2 pixels, and repeat until the end of the line is reached.
<code>fill=value</code>	Specifies a color to fill the oval with which. The argument's value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D  shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> , <code>'blue'</code> , <code>'yellow'</code> , and <code>'cyan'</code> . (If you omit the <code>fill</code> argument, the oval will not be filled.)
<code>outline=value</code>	Specifies the color of the oval's outline. The argument's value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D  shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> ,

	'blue', 'yellow', and 'cyan'. (If you omit the <code>outline</code> argument, the default color is black.)
<code>width=value</code>	Specifies the width of the oval's outline, in pixels. For example, the argument <code>width=5</code> causes the line to be 5 pixels wide. By default, lines are 1 pixel wide.

Program 13-17  demonstrates the `create_oval` method, in lines 13 and 14. The first oval, drawn in line 13, is defined by a rectangle with its upper-left corner at (20, 20) and its lower-right corner at (70, 70). The second oval, drawn in line 14, is defined by a rectangle with its upper-left corner at (100, 100) and its lower-right corner at (180, 130). **Figure 13-32**  shows the window that the program displays.

Figure 13-32 Window displayed by **Program 13-17** 



Program 13-17 (`draw_ovals.py`)

```
1 # This program draws two ovals on a Canvas.
2 import tkinter
```

```

3
4 class MyGUI:
5     def __init__(self):
6         # Create the main window.
7         self.main_window = tkinter.Tk()
8
9         # Create the Canvas widget.
10        self.canvas = tkinter.Canvas(self.main_window,
width=200, height=200)
11
12        # Draw two ovals.
13        self.canvas.create_oval(20, 20, 70, 70)
14        self.canvas.create_oval(100, 100, 180, 130)
15
16        # Pack the canvas.
17        self.canvas.pack()
18
19        # Start the mainloop.
20        tkinter.mainloop()
21
22 # Create an instance of the MyGUI class.
23 my_gui = MyGUI()

```

There are several optional keyword arguments that you can pass to the `create_oval` method. [Table 13-4](#)  lists some of the more commonly used ones.



Tip:

To draw a circle, call the `create_oval` method and make all sides of the bounding rectangle the same length.

Drawing Arcs: The `create_arc` Method

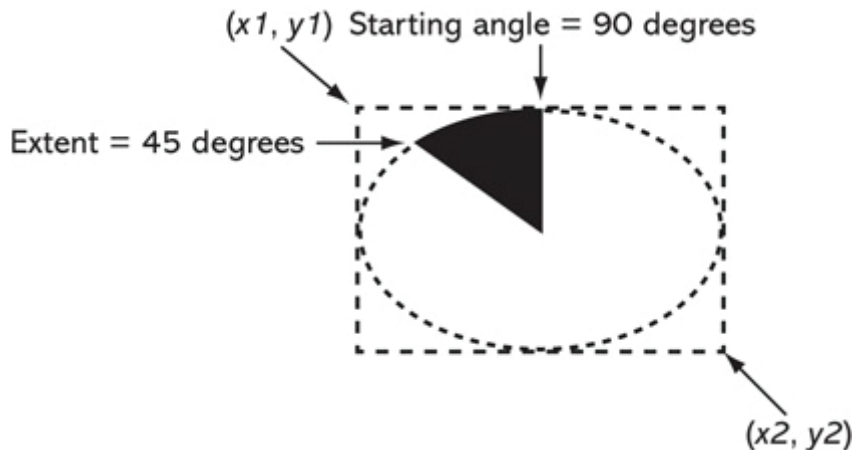
The `create_arc` method draws an arc. Here is the general format of how you call the method:

```
canvas_name.create_arc(x1, y1, x2, y2, start=angle, extent=width,  
options...)
```

This method draws an arc, which is part of an oval. The oval fits just inside a bounding rectangle defined by the coordinates that are passed as arguments. (`x1`, `y1`) are the coordinates of the rectangle's upper-left corner, and (`x2`, `y2`) are the coordinates of the rectangle's lower-right corner. The `start=angle` argument specifies the starting angle, and the `extent=width` argument specifies the counterclockwise

extent of the arc, as an angle. For example, the argument `start=90` specifies that the arc starts at 90 degrees, and the argument `extent=45` specifies that the arc should extend counterclockwise for 45 degrees. This is illustrated in [Figure 13-33](#).

Figure 13-33 Arc properties



In the general format, `options ...` indicates several optional keyword arguments that you may pass to the method. We will examine some of those in [Table 13-5](#).

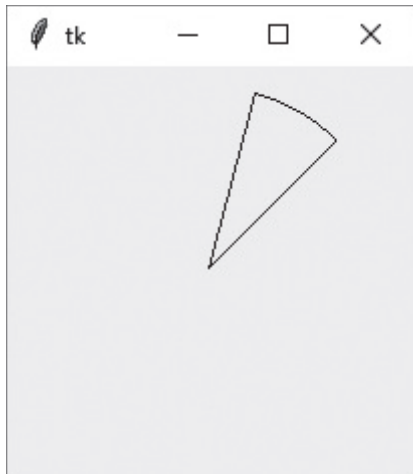
Table 13-5 Some of the optional arguments to the `create_arc` method

Argument	Description
<code>dash=value</code>	This argument causes the outline of the arc to be a dashed line. The value is a tuple, consisting of integers, that specifies a pattern. The first integer specifies the number of pixels to draw, the second integer specifies the number of pixels to skip, and so forth. For example, the argument <code>dash=(5, 2)</code> will draw 5 pixels, skip 2 pixels, and repeat until the end of the line is reached.

<code>fill=value</code>	Specifies a color with which to fill the arc. The argument's value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> , <code>'blue'</code> , <code>'yellow'</code> , and <code>'cyan'</code> . (If you omit the <code>fill</code> argument, the arc will not be filled.)
<code>outline=value</code>	Specifies the color of the arc's outline. The argument's value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> , <code>'blue'</code> , <code>'yellow'</code> , and <code>'cyan'</code> . (If you omit the <code>outline</code> argument, the default color is black.)
<code>style=value</code>	Specifies the style of the arc. The <code>style</code> argument can be one of the values <code>tkinter.PIESLICE</code> , <code>tkinter.ARC</code> , or <code>tkinter.CHORD</code> . See Table 13-6 for more information.
<code>width=value</code>	Specifies the width of the arc's outline, in pixels. For example, the argument <code>width=5</code> causes the line to be 5 pixels wide. By default, lines are 1 pixel wide.

Program 13-18 demonstrates the `create_arc` method. The arc, drawn in line 13, is defined by a bounding rectangle with its upper-left corner at (10, 10), and its lower-right corner at (190, 190). The arc starts at 45 degrees, and extends for 30 degrees. **Figure 13-34** shows the window that the program displays.

Figure 13-34 Window displayed by **Program 13-18**



Program 13-18 *(draw_arc.py)*

```
1  # This program draws an arc on a Canvas.
2  import tkinter
3
4  class MyGUI:
5      def __init__(self):
6          # Create the main window.
7          self.main_window = tkinter.Tk()
8
9          # Create the Canvas widget.
10         self.canvas = tkinter.Canvas(self.main_window,
width=200, height=200)
11
12         # Draw an arc.
13         self.canvas.create_arc(10, 10, 190, 190, start=45,
extent=30)
14
15         # Pack the canvas.
```

```

16         self.canvas.pack()
17
18         # Start the mainloop.
19         tkinter.mainloop()
20
21     # Create an instance of the MyGUI class.
22     my_gui = MyGUI()

```

There are several optional keyword arguments that you can pass to the `create_arc` method. **Table 13-5**  lists some of the more commonly used ones.

There are three styles of arc you can draw with the `style=arcstyle` argument, summarized in **Table 13-6** . (Note the default type is `tkinter.PIESLICE`.) **Figure 13-35**  shows examples of each type of arc.

Table 13-6 Arc types

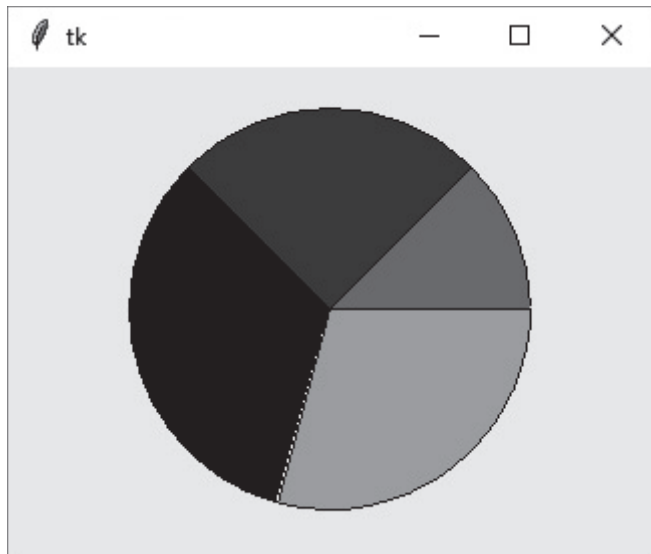
<code>style</code> Argument	Description
<code>style=tkinter.PIESLICE</code>	This is the default arc type. Straight lines will be drawn from each endpoint to the arc's center point. As a result, the arc will be shaped like a pie slice.
<code>style=tkinter.ARC</code>	No lines will connect the endpoints. Only the arc will be drawn.
<code>style=tkinter.CHORD</code>	A straight line will be drawn from one endpoint of the arc to the other endpoint.

Figure 13-35 Types of arcs



Program 13-19  shows an example program that uses arcs to draw a pie chart. The program's output is shown in **Figure 13-36** .

Figure 13-36 Window displayed by Program 13-19 



Program 13-19 *(draw_piechart.py)*

```
1  # This program draws a pie chart on a Canvas.  
2  import tkinter  
3  
4  class MyGUI:  
5      def __init__(self):
```

```
6         self.__CANVAS_WIDTH = 320    # Canvas width
7         self.__CANVAS_HEIGHT = 240    # Canvas height
8         self.__X1 = 60                 # Upper-left X of
bounding rectangle
9         self.__Y1 = 20                 # Upper-left Y of
bounding rectangle
10        self.__X2 = 260                 # Lower-right X of
bounding rectangle
11        self.__Y2 = 220                 # Lower-right Y of
bounding rectangle
12        self.__PIE1_START = 0          # Starting angle of
slice 1
13        self.__PIE1_WIDTH = 45         # Extent of slice 1
14        self.__PIE2_START = 45         # Starting angle of
slice 2
15        self.__PIE2_WIDTH = 90         # Extent of slice 2
16        self.__PIE3_START = 135        # Starting angle of
slice 3
17        self.__PIE3_WIDTH = 120        # Extent of slice 3
18        self.__PIE4_START = 255        # Starting angle of
slice 4
19        self.__PIE4_WIDTH = 105        # Extent of slice 4
20
21        # Create the main window.
22        self.main_window = tkinter.Tk()
23
24        # Create the Canvas widget.
25        self.canvas = tkinter.Canvas(self.main_window,
```

```
26                                     width=self._
   _CANVAS_WIDTH,
27                                     height=self._
   _CANVAS_HEIGHT)
28
29     # Draw slice 1.
30     self.canvas.create_arc(self.__X1, self.__Y1,
self.__X2, self.__Y2,
31                             start=self.__PIE1_START,
32                             extent=self.__PIE1_WIDTH,
33                             fill='red')
34
35     # Draw slice 2.
36     self.canvas.create_arc(self.__X1, self.__Y1,
self.__X2, self.__Y2,
37                             start=self.__PIE2_START,
38                             extent=self.__PIE2_WIDTH,
39                             fill='green')
40
41     # Draw slice 3.
42     self.canvas.create_arc(self.__X1, self.__Y1,
self.__X2, self.__Y2,
43                             start=self.__PIE3_START,
44                             extent=self.__PIE3_WIDTH,
45                             fill='black')
46
47     # Draw slice 4.
48     self.canvas.create_arc(self.__X1, self.__Y1,
```

```

self.__X2, self.__Y2,
49             start=self.__PIE4_START,
50             extent=self.__PIE4_WIDTH,
51             fill='yellow')
52
53     # Pack the canvas.
54     self.canvas.pack()
55
56     # Start the mainloop.
57     tkinter.mainloop()
58
59     # Create an instance of the MyGUI class.
60     my_gui = MyGUI()

```

Let's take a closer look at the `__init__` method in the `MyGUI` class, in **Program 13-19** :

- Lines 6 and 7 define attributes for the `Canvas` widget's width and height.
- Lines 8–11 define attributes for the coordinates of the upper-left and lower-right corners of the bounding rectangle that each arc will share.
- Lines 12–19 define attributes for each of the pie slice's starting angle and extent.
- Line 22 creates the main window, and lines 25–27 create the `Canvas` widget.
- Lines 30–33 create the first pie slice, setting its fill color to red.

- Lines 36–39 create the second pie slice, setting its fill color to green.
- Lines 42–45 create the third pie slice, setting its fill color to black.
- Line 54 packs the `Canvas`, making its contents visible, and line 57 starts the `tkinter` module's `mainloop` function.

Drawing Polygons: The `create_polygon` Method

The `create_polygon` method draws a closed polygon on the `Canvas`. A polygon is constructed of multiple line segments that are connected. The point where two line segments are connected is called a *vertex*. Here is the general format of how you call the method to draw a polygon:

```
canvas_name.create_polygon(x1, y1, x2, y2, ..., options ...)
```

The arguments `x1` and `y1` are the (X, Y) coordinates of the first vertex, `x2` and `y2` are the (X, Y) coordinates of the second vertex, and so on. The method will automatically close the polygon by drawing a line segment from the last vertex to the first vertex. In the general format, `options ...` indicates several optional keyword arguments that you may pass to the method. We will examine some of those in [Table 13-7](#) .

Table 13-7 Some of the optional arguments to the `create_polygon` method

Argument	Description
<code>dash=value</code>	This argument causes the outline of the polygon to be a dashed line. The first integer specifies the number of pixels to draw, the second integer specifies the number of pixels to skip, and so forth. For example, the argument <code>dash=(5, 2)</code> will draw 5 pixels, skip 2 pixels, and repeat until the end of the line is reached.
<code>fill=value</code>	Specifies a color with which to fill the polygon. The value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D  shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> , <code>'blue'</code> , <code>'yellow'</code> , and <code>'cyan'</code> . (If you omit the <code>fill</code> argument, the polygon will be filled with black.)
<code>outline=value</code>	Specifies the color of the polygon's outline. The value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D  shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> , <code>'blue'</code> , <code>'yellow'</code> , and <code>'cyan'</code> . (If you omit the <code>outline</code> argument, the default color is black.)
<code>smooth=value</code>	By default, the <code>smooth</code> argument is set to <code>False</code> , which makes the method draw straight lines connecting the specified points. If you specify <code>smooth=True</code> , the lines are drawn as curved splines.
<code>width=value</code>	The value is an integer that specifies the width of the polygon's outline, in pixels. For example, the argument <code>width=5</code> causes the outline to be 5 pixels wide. By default, the outline is 1 pixel wide.

Program 13-20  demonstrates the `create_polygon` method. The statement in lines 13 and 14 draws a polygon with eight vertices. The first vertex is at (60, 20), the second vertex is at (100, 20), and so on, as illustrated in **Figure 13-37** . **Figure 13-38**  shows the window that the program displays.

Figure 13-37 Points of each vertex in the polygon

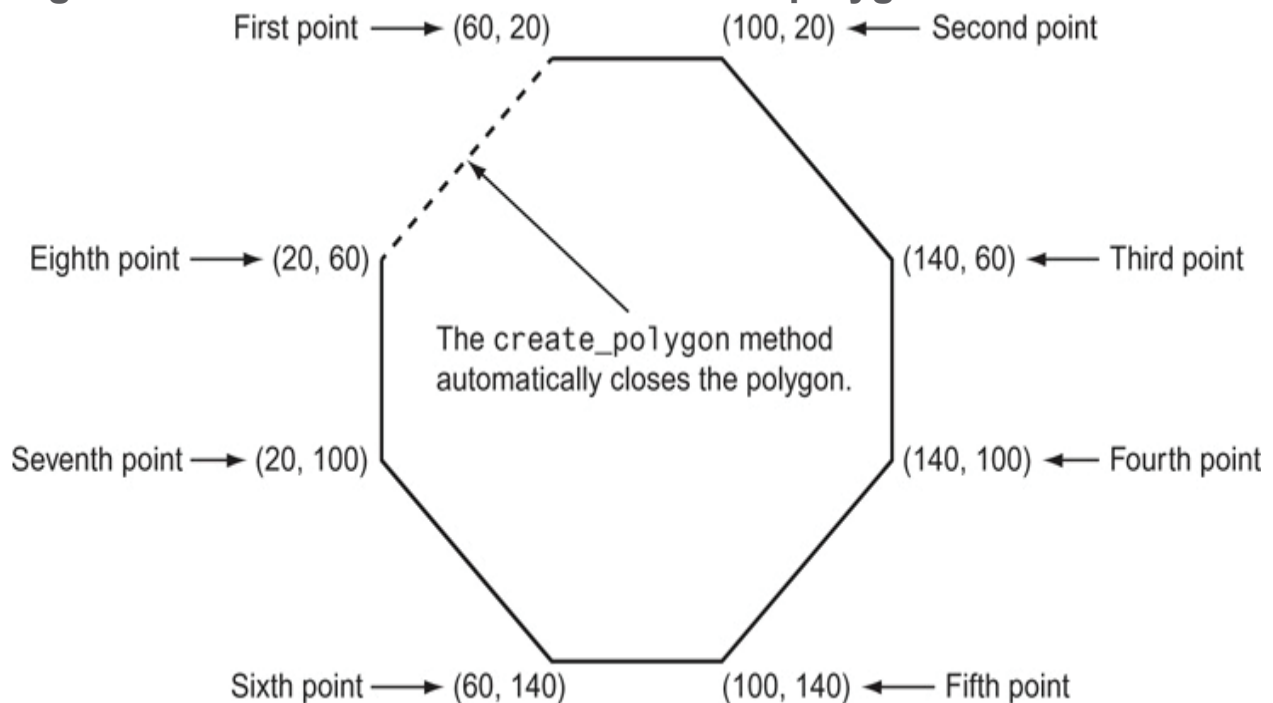
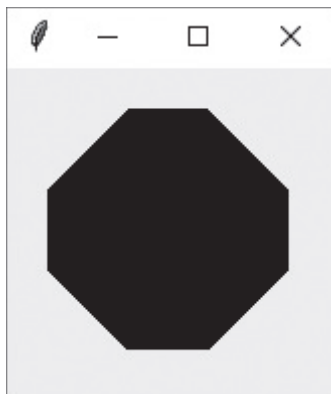


Figure 13-38 Window displayed by Program 13-20 



Program 13-20 *(draw_polygon.py)*

```
1  # This program draws a polygon on a Canvas.
2  import tkinter
3
4  class MyGUI:
5      def __init__(self):
6          # Create the main window.
7          self.main_window = tkinter.Tk()
8
9          # Create the Canvas widget.
10         self.canvas = tkinter.Canvas(self.main_window,
width=160, height=160)
11
12         # Draw a polygon.
13         self.canvas.create_polygon(60, 20, 100, 20, 140,
60, 140, 100,
14                                     100, 140, 60, 140, 20,
100, 20, 60)
15
16         # Pack the canvas.
17         self.canvas.pack()
18
19         # Start the mainloop.
20         tkinter.mainloop()
21
22     # Create an instance of the MyGUI class.
23     my_gui = MyGUI()
```


There are several optional keyword arguments that you can pass to the `create_polygon` method. [Table 13-7](#) lists some of the more commonly used ones.

Drawing Text: The `create_text` Method

You can use the `create_text` function to display text on the `Canvas`. Here is the general format of how you call the method:

```
canvas_name.create_text(x, y, text=text, options ...)
```

The arguments `x` and `y` are the (X, Y) coordinates of the text's insertion point, and the `text=text` argument specifies the text to display. By default, the text is centered horizontally and vertically around the insertion point. In the general format, `options ...` indicates several optional keyword arguments that you may pass to the method. We will examine some of those in [Table 13-8](#).

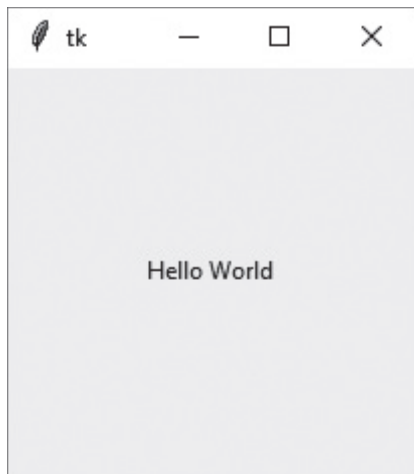
Table 13-8 Some of the optional arguments to the `create_text` method

Argument	Description
----------	-------------

<code>anchor=value</code>	This argument specifies how the text is positioned relative to its insertion point. By default, the <code>anchor</code> argument is set to <code>tkinter.CENTER</code> , causing the text to be centered both vertically and horizontally around the insertion point. You can specify any of the values listed in Table 13-9 .
<code>fill=value</code>	Specifies the text color. The value is the name of a color, as a string. There are numerous predefined color names that you can use, and Appendix D shows the complete list. Some of the more common colors are <code>'red'</code> , <code>'green'</code> , <code>'blue'</code> , <code>'yellow'</code> , and <code>'cyan'</code> . (If you omit the <code>fill</code> argument, the text will be black.)
<code>font=value</code>	To change the default font, create a <code>tkinter.font.Font</code> object and pass it as value of the <code>font</code> argument. (See the discussion on fonts later in this section.)
<code>justify=value</code>	If multiple lines of text are displayed, this argument specifies how the lines are justified. The values can be <code>tk.LEFT</code> , <code>tk.CENTER</code> , or <code>tk.RIGHT</code> . The default value is <code>tk.LEFT</code> .

Program 13-21 demonstrates the `create_text` method. The statement in line 13 displays the text `'Hello World'` in the center of the window, at coordinates (100, 100). **Figure 13-39** shows the window that the program displays.

Figure 13-39 Window displayed by **Program 13-21**



Program 13-21 *(draw_text.py)*

```
1  # This program draws text on a Canvas.
2  import tkinter
3
4  class MyGUI:
5      def __init__(self):
6          # Create the main window.
7          self.main_window = tkinter.Tk()
8
9          # Create the Canvas widget.
10         self.canvas = tkinter.Canvas(self.main_window,
width=200, height=200)
11
12         # Display text in the center of the window.
13         self.canvas.create_text(100, 100, text='Hello
World')
14
15         # Pack the canvas.
```

```

16         self.canvas.pack()
17
18         # Start the mainloop.
19         tkinter.mainloop()
20
21     # Create an instance of the MyGUI class.
22     my_gui = MyGUI()

```

There are several optional keyword arguments that you can pass to the `create_text` method. [Table 13-8](#) lists some of the more commonly used ones.

There are nine different ways that you can position the text relative to its insertion point. You use the `anchor=position` argument to change the positioning. The different values of the argument are summarized in [Table 13-9](#). Note the default value is `tkinter.CENTER`.

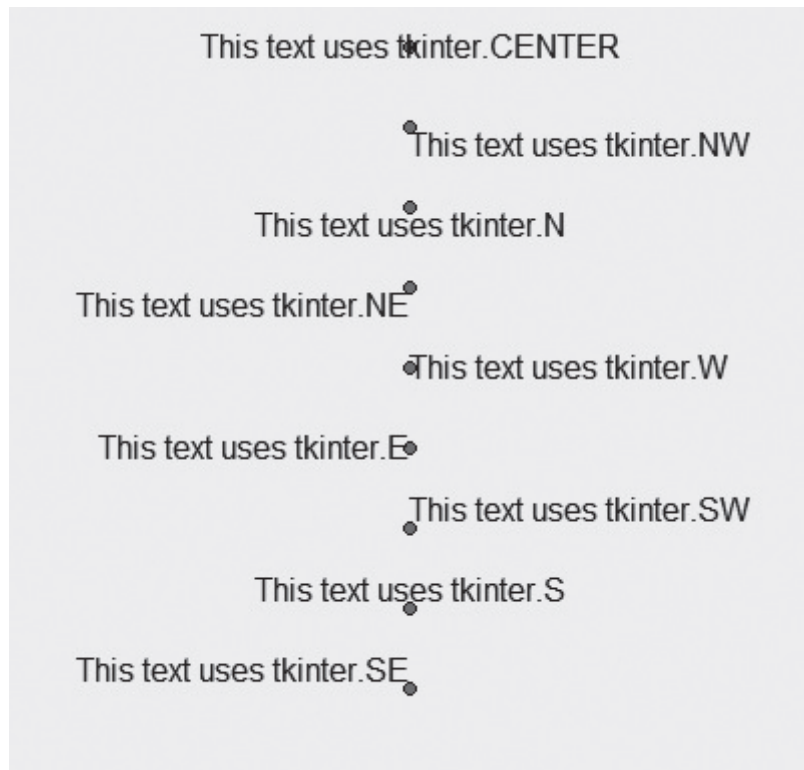
Table 13-9 `anchor` values

<code>anchor</code> Argument	Description
<code>anchor=tkinter.CENTER</code>	Causes the text to be vertically and horizontally centered around the insertion point. This is the default positioning.
<code>anchor=tkinter.NW</code>	Causes the text to be positioned, so the insertion point is at the text's upper-left corner (northwest).
<code>anchor=tkinter.N</code>	Causes the text to be positioned, so the insertion point is centered along the top edge of the text (north).

<code>anchor=tkinter.NE</code>	Causes the text to be positioned, so the insertion point is at the text's upper-right corner (northeast).
<code>anchor=tkinter.W</code>	Causes the text to be positioned, so the insertion point is at the text's left edge, in the middle (west).
<code>anchor=tkinter.E</code>	Causes the text to be positioned, so the insertion point is at the text's right edge, in the middle (east).
<code>anchor=tkinter.SW</code>	Causes the text to be positioned, so the insertion point is at the text's lower-left corner (southwest).
<code>anchor=tkinter.S</code>	Causes the text to be positioned, so the insertion point is centered along the bottom edge of the text (south).
<code>anchor=tkinter.SE</code>	Causes the text to be positioned, so the insertion point is at the text's lower-right corner (southeast).

Figure 13-40  illustrates the different anchor positions. In the figure, each line of text has a dot that represents the position of the insertion point.

Figure 13-40 Results of the various anchor values



Setting the Font

You can set the font that is used with the `create_text` method by creating a `Font` object, and passing it as the `font=` argument. The `Font` class is in the `tkinter.font` module, so you must include the following `import` statement in your program:

```
import tkinter.font
```

Here is an example of creating a `Font` object that specifies a Helvetica 12-point font:

```
myfont = tkinter.font.Font(family='Helvetica', size='12')
```

When you construct a `Font` object, you can pass values for any of the keyword arguments shown in [Table 13-10](#) .

Table 13-10 Keyword arguments for the `Font` class

Argument	Description
<code>family=value</code>	This argument is a string that specifies the name of the font family, such as <code>'Arial'</code> , <code>'Courier'</code> , <code>'Helvetica'</code> , <code>'Times New Roman'</code> , and so on.
<code>size=value</code>	This argument is an integer that specifies the font size in points.
<code>weight=value</code>	This argument specifies the weight of the font. The valid values are the strings <code>'bold'</code> and <code>'normal'</code> .
<code>slant=value</code>	This argument specifies the slant of the font. If you want the font to appear slanted, specify <code>'italic'</code> . If you want the font to appear unslanted, specify <code>'roman'</code> .
<code>underline=value</code>	If you want the text to appear underlined, specify 1. Otherwise, specify 0.
<code>overstrike=value</code>	If you want the text to appear crossed-out, specify 1. Otherwise, specify 0.

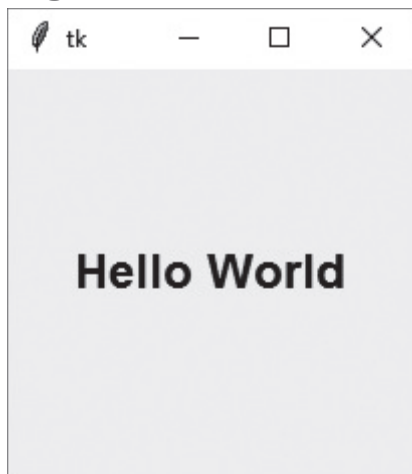
The names of the font families that are available are somewhat system-dependent. To see a list of the font families that you have

installed, enter the following at the Python shell:

```
>>> import tkinter
>>> import tkinter.font
>>> tkinter.Tk()
<tkinter.Tk object .>
>>> tkinter.font.families()
```

Program 13-22  shows an example of displaying text with an 18-point boldface Helvetica font. **Figure 13-41**  shows the window that the program displays.

Figure 13-41 Window displayed by **Program 13-22** 



Program 13-22 `(font_demo.py)`

```
1 # This program draws text on a Canvas.
2 import tkinter
3 import tkinter.font
```



```
4
5 class MyGUI:
6     def __init__(self):
7         # Create the main window.
8         self.main_window = tkinter.Tk()
9
10        # Create the Canvas widget.
11        self.canvas = tkinter.Canvas(self.main_window,
width=200, height=200)
12
13        # Create a Font object.
14        myfont = tkinter.font.Font(family='Helvetica',
size=18, weight='bold')
15
16        # Display some text.
17        self.canvas.create_text(100, 100, text='Hello
World', font=myfont)
18
19        # Pack the canvas.
20        self.canvas.pack()
21
22        # Start the mainloop.
23        tkinter.mainloop()
24
25 # Create an instance of the MyGUI class.
26 my_gui = MyGUI()
```



Checkpoint

13.19 In the `Canvas` widget's screen coordinate system, what are the coordinates of the pixel in the upper-left corner of the window?

13.20 Using the `Canvas` widget's screen coordinate system with a window that is 640 pixels wide by 480 pixels high, what are the coordinates of the pixel in the lower-right corner?

13.21 How is the `Canvas` widget's screen coordinate system different from the Cartesian coordinate system used by the turtle graphics library?

13.22 What Canvas widget methods would you use to draw each of the following types of shapes?

- a. A circle
- b. A square
- c. A rectangle
- d. A closed six-sided shape
- e. An ellipse
- f. An arc

Review Questions

Multiple Choice

1. The _____ is the part of a computer with which the user interacts.
 - a. central processing unit
 - b. user interface
 - c. control system
 - d. interactivity system

2. Before GUIs became popular, the _____ interface was the most commonly used.
 - a. command line
 - b. remote terminal
 - c. sensory
 - d. event-driven

3. A _____ is a small window that displays information and allows the user to perform actions.
 - a. menu
 - b. confirmation window
 - c. startup screen
 - d. dialog box

4. These types of programs are event driven.
- a. command line
 - b. text-based
 - c. GUI
 - d. procedural
5. An item that appears in a program's graphical user interface is known as a(n) _____.
- a. gadget
 - b. widget
 - c. tool
 - d. iconified object
6. You can use this module in Python to create GUI programs.
- a. `GUI`
 - b. `PythonGui`
 - c. `tkinter`
 - d. `tgui`
7. This widget is an area that displays one line of text.
- a. `Label`
 - b. `Entry`
 - c. `TextLine`
 - d. `Canvas`
8. This widget is an area in which the user may type a single line of input from the keyboard.

- a. `Label`
- b. `Entry`
- c. `TextLine`
- d. `Input`

9. This widget is a container that can hold other widgets.

- a. `Grouper`
- b. `Composer`
- c. `Fence`
- d. `Frame`

10. This method arranges a widget in its proper position, and it makes the widget visible when the main window is displayed.

- a. `pack`
- b. `arrange`
- c. `position`
- d. `show`

11. A(n) _____ is a function or method that is called when a specific event occurs.

- a. callback function
- b. auto function
- c. startup function
- d. exception

12. The `showinfo` function is in this module.

- a. `tkinter`
- b. `tkinfo`
- c. `sys`
- d. `tkinter.messagebox`

13. You can call this method to close a GUI program.

- a. The root widget's `destroy` method
- b. Any widget's `cancel` method
- c. The `sys.shutdown` function
- d. The `Tk.shutdown` method

14. You call this method to retrieve data from an `Entry` widget.

- a. `get_entry`
- b. `data`
- c. `get`
- d. `retrieve`

15. An object of this type can be associated with a `Label` widget, and any data stored in the object will be displayed in the `Label`.

- a. `StringVar`
- b. `LabelVar`
- c. `LabelValue`
- d. `DisplayVar`

16. If there are a group of these in a container, only one of them can be selected at any given time.

- a. `Checkbutton`
- b. `Radiobutton`
- c. `Mutualbutton`
- d. `Button`

17. The _____ widget provides methods for drawing simple 2D shapes.

- a. `Shape`
- b. `Draw`
- c. `Palette`
- d. `Canvas`

True or False

1. The Python language has built-in keywords for creating GUI programs.
2. Every widget has a `quit` method that can be called to close the program.
3. The data that you retrieve from an `Entry` widget is always of the `int` data type.
4. A mutually exclusive relationship is automatically created among all `Radiobutton` widgets in the same container.
5. A mutually exclusive relationship is automatically created among all `Checkbutton` widgets in the same container.

Short Answer

1. When a program runs in a text-based environment, such as a command line interface, what determines the order in which things happen?
2. What does a widget's `pack` method do?
3. What does the `tkinter` module's `mainloop` function do?
4. If you create two widgets and call their `pack` methods with no arguments, how will the widgets be arranged inside their parent widget?
5. How do you specify that a widget should be positioned as far left as possible inside its parent widget?
6. How do you retrieve data from an `Entry` widget?
7. How can you use a `StringVar` object to update the contents of a `Label` widget?
8. How can you use an `IntVar` object to determine which `Radiobutton` has been selected in a group of `Radiobuttons`?
9. How can you use an `IntVar` object to determine whether a `Checkbutton` has been selected?

Algorithm Workbench

1. Write a statement that creates a `Label` widget. Its parent should be `self.main_window`, and its text should be `'Programming is`


```
fun!'
```

2. Assume `self.label1` and `self.label2` reference two `Label` widgets. Write code that packs the two widgets so they are positioned as far left as possible inside their parent widget.
3. Write a statement that creates a `Frame` widget. Its parent should be `self.main_window`.
4. Write a statement that displays an info dialog box with the title “Program Paused” and the message “Click OK when you are ready to continue.”
5. Write a statement that creates a `Button` widget. Its parent should be `self.button_frame`, its text should be `'Calculate'`, and its callback function should be the `self.calculate` method.
6. Write a statement that creates a `Button` widget that closes the program when it is clicked. Its parent should be `self.button_frame`, and its text should be `'Quit'`.
7. Assume the variable `data_entry` references an `Entry` widget. Write a statement that retrieves the data from the widget, converts it to an `int`, and assigns it to a variable named `var`.
8. Assume that in a program, the following statement creates a `Canvas` widget and assigns it to the `self.canvas` variable:

```
self.canvas = tkinter.Canvas(self.main_window, width=200,  
height=200)
```

Write statements that do the following:

- a. Draws a blue line from the `Canvas` widget's upper-left corner to its lower-right corner. The line should be 3 pixels wide.
- b. Draws a rectangle with a red outline and a black interior. The rectangle's corners should appear in the following locations on the `Canvas`:
 - Upper-left: (50, 50)
 - Upper-right: (100, 50)
 - Lower-left: (50, 100)
 - Lower-right: (100, 100)
- c. Draws a green circle. The circle's center-point should be at (100, 100), and its radius should be 50.
- d. Draws a blue-filled arc that is defined by a bounding rectangle with its upper-left corner at (20, 20), and its lower-right corner at (180, 180). The arc should start at 0 degrees, and extend for 90 degrees.

Programming Exercises

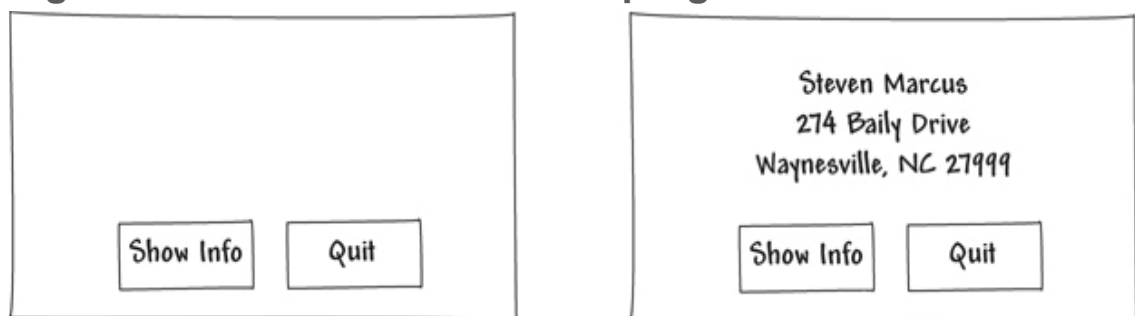
1. Name and Address



The Name and Address Problem

Write a GUI program that displays your name and address when a button is clicked. The program's window should appear as the sketch on the left side of [Figure 13-42](#)  when it runs. When the user clicks the *Show Info* button, the program should display your name and address, as shown in the sketch on the right of the figure.

Figure 13-42 Name and address program



2. Latin Translator

Look at the following list of Latin words and their meanings:

Latin	English
sinister	left
dexter	right
medium	center

Write a GUI program that translates the Latin words to English. The window should have three buttons, one for each Latin word. When the user clicks a button, the program displays the English translation in a label.

3. Miles Per Gallon Calculator

Write a GUI program that calculates a car's gas mileage. The program's window should have `Entry` widgets that let the user enter the number of gallons of gas the car holds, and the number of miles it can be driven on a full tank. When a *Calculate MPG* button is clicked, the program should display the number of miles that the car may be driven per gallon of gas. Use the following formula to calculate miles-per-gallon:
$$\text{MPG} = \frac{\text{miles}}{\text{gallons}}$$

4. Celsius to Fahrenheit

Write a GUI program that converts Celsius temperatures to Fahrenheit temperatures. The user should be able to enter a Celsius temperature, click a button, then see the equivalent

Fahrenheit temperature. Use the following formula to make the conversion:

$$F=95C+32$$

F is the Fahrenheit temperature, and *C* is the Celsius temperature.

5. **Property Tax**

A county collects property taxes on the assessment value of property, which is 60 percent of the property's actual value. If an acre of land is valued at \$10,000, its assessment value is \$6,000. The property tax is then \$0.75 for each \$100 of the assessment value. The tax for the acre assessed at \$6,000 will be \$45.00. Write a GUI program that displays the assessment value and property tax when a user enters the actual value of a property.

6. **Joe's Automotive**

Joe's Automotive performs the following routine maintenance services:

- Oil change—\$30.00
- Lube job—\$20.00
- Radiator flush—\$40.00
- Transmission flush—\$100.00
- Inspection—\$35.00
- Muffler replacement—\$200.00
- Tire rotation—\$20.00

Write a GUI program with check buttons that allow the user to select any or all of these services. When the user clicks a

button, the total charges should be displayed.

7. Long-Distance Calls

A long-distance provider charges the following rates for telephone calls:

Rate Category	Rate per Minute
Daytime (6:00 A.M. through 5:59 P.M.)	\$0.07
Evening (6:00 P.M. through 11:59 P.M.)	\$0.12
Off-Peak (midnight through 5:59 A.M.)	\$0.05

Write a GUI application that allows the user to select a rate category (from a set of radio buttons), and enter the number of minutes of the call into an `Entry` widget. An info dialog box should display the charge for the call.

8. This Old House

Use the `Canvas` widget that you learned in this chapter to draw a house. Be sure to include at least two windows and a door. Feel free to draw other objects as well, such as the sky, sun, and even clouds.

9. Tree Age

Counting the growth rings of a tree is a good way to tell the age of a tree. Each growth ring counts as one year. Use a `Canvas` widget to draw how the growth rings of a 5-year-old tree might look. Then, using the `create_text` method, number each growth ring starting from the center and working outward with the age in years associated with that ring.

10. Hollywood Star

Make your own star on the Hollywood Walk of Fame. Write a program that displays a star similar to the one shown in **Figure 13-43** , with your name displayed in the star.

Figure 13-43 Hollywood star



11. **Vehicle Outline**

Using the shapes you learned about in this chapter, draw the outline of the vehicle of your choice (car, truck, airplane, and so forth).

12. **Solar System**

Use a `Canvas` widget to draw each of the planets of our solar system. Draw the sun first, then each planet according to distance from the sun (Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune, and the dwarf planet, Pluto). Label each planet using the `create_text` method.

Appendix A Installing Python

Downloading Python

To run the programs shown in this book, you will need to install Python 3.0, or a later version. You can download the latest version of Python from www.python.org/downloads. This appendix discusses installing Python for Windows. Python is also available for the Mac, Linux, and several other platforms. Links to download versions of Python for these systems are shown on the Python download site at www.python.org/downloads.



Keep in mind that there are two *families* of Python that you can download: Python 3.x and Python 2.x. The programs in this book work only with the Python 3.x family.